Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Célula robotizada para control de calidad mediante inspección visual

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Gómez Buleo, Álvaro

Tutor: Vendrell Vidal, Eduardo

[2025/2026]

Resumen

Este proyecto propone el diseño y programación de una célula robotizada encargada de la inspección visual de objetos, ya sean en 2D o en 3D, con el objeto de realizar el control de calidad de estos en un proceso de producción. La tarea del sistema se basa en la detección de defectos en piezas basándose en un sistema de visión artificial, que determinará si los objetos presentan algún defecto, enviando la señal para que sean clasificados como correctos o defectuosos.

Esta célula está compuesta por un robot UR5e con una cámara OnRobot Eyes montada en su efector final, con el fin de programar las acciones necesarias para realizar la inspección de calidad propuesta mediante la toma de imágenes.

Para la realización de este proyecto, se usará el software RoboDK, en el que se diseñará y se programará la célula, incluyendo la programación del propio robot, los algoritmos de procesamiento de imágenes, y la conexión entre los distintos elementos de la célula. Se llevará a cabo mediante una simulación basada en el concepto del gemelo digital, por lo que se creará una simulación del entorno físico de la planta en el modelo virtual.

Palabras clave: Robótica, Visión Artificial, Automatización, Robótica Industrial, Programación, Python, RoboDK

Abstract

This project proposes the design and programming of a robotic cell responsible for the visual inspection of both 2D and 3D objects, aiming to perform quality control within a production process. The system's task is based on defect detection in parts using a computer vision system, which determines whether the objects have any flaws and sends a signal to classify them as accepted or defective.

This cell consists of a UR5e robot with an OnRobot Eyes camera mounted on its end effector to program the necessary actions for the proposed quality inspection through image acquisition.

To carry out this project, RoboDK software will be used to design and program the cell, including the programming of the robot itself, the image processing algorithms, and communication between the different elements of the cell. The project will be developed through a simulation based on the digital twin concept, creating a simulation of the physical plant environment within the virtual model.

Keywords: Robotics, Computer Vision, Automation, Industrial Robotics, Programming, Python, RoboDK

Tabla de contenidos

Índice

1. Introducción 6
1.1. Introducción y motivación 6
1.2. Objetivos 7
1.3. Aplicabilidad 8
1.4. Descripción y funcionamiento del sistema automatizado 9
2. Estado del arte y antecedentes 10
2.1. Automatización industrial y células robotizadas 10
2.2. Robótica colaborativa en procesos industriales 11
2.3. Sistemas de visión artificial para inspección industrial 12
2.4. Integración de visión artificial y robótica 13
2.5. Técnicas de procesamiento de imágenes aplicadas a la inspección 14
2.6. Simulación y gemelos digitales en robótica industrial 16
2.6.1. El concepto de gemelo digital 16
2.6.2. RoboDK como herramienta de simulación 17
2.6.3. Uso de los gemelos digitales en la robótica industrial 18
2.7. Tecnologías y soluciones comerciales existentes 19
3. Metodología 20
3.1. Elección de los sistemas para el proyecto 20
3.1.1. Elección del robot 20
3.1.2. Elección de la herramienta de simulación 22
3.1.3. Elección de la cámara 22
3.2. Diseño de las piezas a escanear 23
3.3. Diseño del proceso y el layout 25
3.3.1. Diseño del proceso 25
3.3.2. Diseño del layout 26
3.4. Estudio de seguridad industrial 28
3.4.1. Marco normativo 28
3.4.2. Cumplimiento de la normativa 30
3.5. Estudio de ciberseguridad 33
3.5.1. Normativa aplicable 33
3.5.2. Cumplimiento de la normativa 34

3.6. Integración en el simulador 35
3.7. Calibración de la cámara y programación del procesamiento de imágenes ..40
3.7.1. Procesamiento de las plantillas 40
3.7.2. Procesamiento de los cubos 42
3.7.3. Procesamiento de las tazas 44
3.8. Programación del robot, la mesa y las cintas 46
3.8.1. Programación del robot 46
3.8.2. Programación de la mesa 48
3.8.3. Programación de las cintas 48
3.9. Sincronización y funcionamiento de la célula al completo 49
3.10. Planificación temporal 51
3.10.1. Planificación temporal del proyecto 51
3.10.2. Planificación temporal de instalación en planta 53
4. Estudio económico 55
4.1. Costes de material 55
4.2. Costes de ingeniería e instalación 56
4.3. Análisis coste-beneficio 56
5. Conclusiones y líneas futuras 59
5.1. Conclusiones 59
5.2. Posibles líneas futuras 61
6. Bibliografía 62
Anexos 64
Anexo I: Compromiso con los Objetivos de Desarrollo Sostenible: 64

1. Introducción

1.1. Introducción y motivación

La creciente competitividad del sector industrial ha impulsado la creación de tecnologías que permitan aumentar la productividad, mejorar la calidad de los productos y reducir los costes asociados a los procesos de fabricación. En este contexto, la automatización industrial se ha convertido en un elemento fundamental dentro de la industria moderna, permitiendo sustituir tareas repetitivas y susceptibles al error humano por sistemas automáticos capaces de trabajar de forma continua, precisa y fiable.

Uno de los aspectos más relevantes dentro de cualquier proceso productivo es el control de calidad. Tradicionalmente, la inspección de piezas ha sido realizada de forma manual por operarios especializados, lo que implica limitaciones relacionadas con la fatiga, la subjetividad en la evaluación y la dificultad para mantener un nivel constante de precisión. Como consecuencia, en los últimos años han adquirido una gran importancia los sistemas de visión artificial integrados con robots industriales, capaces de inspeccionar productos de manera automática y objetiva.

La combinación de la robótica industrial con técnicas de procesamiento de imágenes permite el desarrollo de células robotizadas capaces de detectar defectos, verificar dimensiones, comprobar ensamblajes o clasificar piezas según diferentes criterios de calidad. Estos sistemas no solo incrementan la velocidad de inspección, sino que también mejoran la repetibilidad del proceso y facilitan la integración dentro de líneas de producción automatizadas.

Además, el desarrollo de herramientas de simulación y programación offline ha favorecido la implantación de este tipo de soluciones. Mediante el uso del concepto del gemelo digital es posible diseñar, programar y validar una célula robotizada antes de su implementación física, reduciendo tiempos de desarrollo, costes de puesta en marcha y posibles errores durante la integración del sistema.

En este tipo de aplicaciones, la integración de los diferentes sistemas que componen una célula automatizada supone uno de los principales retos durante su desarrollo. El correcto funcionamiento del sistema no depende únicamente del desempeño individual del robot o del sistema de visión, sino también de la coordinación entre los distintos elementos que intervienen en el proceso. Por este motivo, resulta necesario disponer de herramientas que permitan analizar y validar el comportamiento conjunto de la célula, comprobando aspectos como los movimientos del robot, los tiempos de operación, el flujo de piezas y la comunicación entre los diferentes dispositivos antes de realizar una posible implementación física.

La flexibilidad constituye además una característica especialmente relevante en los sistemas de inspección automatizada. A diferencia de una solución diseñadas para un único producto específico, los sistemas basados en visión artificial pueden adaptarse mediante cambios en los algoritmos de procesamiento y en los parámetros de inspección a diferentes tipos de piezas o criterios de calidad. Esta capacidad de adaptación permite plantear soluciones más versátiles, capaces de responder a las

necesidades de diferentes procesos productivos sin requerir necesariamente modificaciones significativas en la instalación.

En este proyecto se plantea el diseño y programación de una célula robotizada destinada a la inspección visual de piezas mediante visión artificial. La estación, desarrollada en el entorno de simulación RoboDK, está compuesta por un robot colaborativo Universal Robots UR5e equipado con una cámara OnRobot Eyes instalada en el efector final, así como por un sistema de transporte mediante cintas transportadoras que permite la entrada de piezas a inspeccionar y la separación automática entre piezas correctas y defectuosas. La simulación reproduce el funcionamiento de una instalación industrial real siguiendo el concepto del gemelo digital, permitiendo validar tanto la programación del robot como los algoritmos de procesamiento de imágenes y la comunicación entre los diferentes elementos de la célula.

El objetivo principal del proyecto es desarrollar un sistema automatizado capaz de inspeccionar objetos bidimensionales y tridimensionales, detectar posibles defectos mediante técnicas de visión artificial y clasificar las piezas según el resultado de la inspección. Con ello se pretende demostrar la viabilidad de integrar robótica industrial, programación y visión artificial en un entorno de fabricación automatizado, contribuyendo a mejorar la eficiencia y la calidad de los procesos de producción.

1.2. Objetivos

Los objetivos que lograr durante el desarrollo de este proyecto para cumplir con el objetivo principal son los siguientes:

- **El diseño y la elección de los elementos de la célula,** incluyendo desde los elementos de seguridad, como el vallado que protege la célula de automatización, y los elementos de transporte de los productos, hasta el propio robot encargado de la tarea, incluyendo la cámara que usará para ello. En este objetivo se busca conseguir el layout completo de la célula, incluyendo el posicionamiento de la cinta de entrada y las dos cintas de salida de productos ya comprobados, y la ubicación del robot y la cinta rotatoria. Una vez diseñado el layout, se recreará en el entorno de RoboDK, lo que permitirá comprobar que los elementos seleccionados son adecuados para la realización de esta tarea, comparar los tiempos de inspección y las distintas métricas que permiten calcular el beneficio obtenido mediante la instalación de esta automatización, y comprobar que no se generan conflictos con la interacción entre los distintos elementos, comprobando así si son válidos para la implementación en la planta industrial física.

- **La programación del robot para el funcionamiento de la célula:** En este proyecto, el robot será programado en la simulación de RoboDK, el cual incluye su propio sistema integrado de programación de robots, pero para permitir la comunicación entre distintos procesos, y mejorar la calidad de la simulación, la programación del robot, del sistema de visión artificial y reconocimiento de objetos, de las cintas de entrada y salida y de la cinta rotatoria, serán realizados utilizando la API de Python de RoboDK. Esta API permite la escritura de programas en Python que pueden ser ejecutados en la propia aplicación.

Además, el uso de scripts de Python permite la coordinación del funcionamiento de los diferentes elementos de la célula, facilitando el intercambio de información entre ellos y permitiendo establecer una secuencia de trabajo automatizada. De esta forma, el robot podrá recibir información del sistema de visión, determinar las acciones necesarias y coordinar sus movimientos con el funcionamiento de las cintas y demás elementos de la célula. Además, mediante el uso de la conexión al robot que incluye RoboDK, se podría ejecutar el programa directamente en el robot UR conectándolo mediante red a la estación simulada.

- **Programación de la cámara y sus algoritmos de detección:** En este objetivo se incluye la programación de la cámara, incluyendo los algoritmos de detección de objetos defectuosos. Esto también será realizado mediante el uso de la API de Python para RoboDK y de librerías de Python como *NumPy* y *OpenCV (cv2)*. Se creará un script para la comprobación de cada uno de los tipos de objeto a escanear, los cuales funcionan mediante la toma de capturas o *snapshots*, que serán comparados con imágenes modelo de su respectivo producto, y mediante algoritmos de detección de bordes, podrán detectar si un objeto es válido o defectuoso. Estos algoritmos son válidos tanto en superficies planas como curvas y son fácilmente ajustables para una mayor o menor tolerancia según requiera la aplicación.

1.3. Aplicabilidad

La solución desarrollada en este proyecto presenta una aplicación directa en entornos industriales en los que sea necesario realizar procesos de inspección y clasificación de productos de forma automatizada. La integración de un sistema de visión artificial con un robot industrial permite automatizar tareas de control de calidad que pueden requerir una intervención manual continua. De esta forma, se pretende aumentar la capacidad de inspección, reducir la influencia del error humano y mantener unos criterios de evaluación homogéneos a lo largo del proceso productivo.

Este tipo de solución es especialmente aplicable en líneas de producción en las que se manipulen diferentes clases de producto y en las que sea necesario comprobar de forma sistemática que cada pieza cumple con unos requisitos de calidad. El sistema de visión artificial permite realizar inspecciones de forma rápida y repetitiva, mientras que el robot permite el escaneo de objetos más complejos además de la fácil adaptación a posibles nuevos objetos. Además, la cinta se ocupa del clasificado de los objetos una vez escaneados, lo que hace mucho más rápida la ejecución de la tarea. La automatización de estas operaciones permite liberar al operario de tareas repetitivas, pudiendo dedicar sus recursos a labores de supervisión, mantenimiento o intervención ante posibles incidencias.

Una de las principales ventajas de la solución propuesta es su capacidad de adaptación a diferentes productos y criterios de inspección. Los algoritmos de procesamiento de imágenes se pueden modificar de forma sencilla para trabajar con diferentes características de las piezas o establecer distintos niveles de tolerancia. Del mismo modo, el posicionamiento del robot para la toma de imágenes y la distribución de los sistemas de transporte puede adaptarse para modificar la secuencia de trabajo o

incorporar nuevos objetos. Esta flexibilidad permite plantear la célula como una solución reutilizable y ampliable frente a sistemas de inspección diseñados específicamente para una única aplicación.

La utilización de un entorno de simulación como RoboDK también amplía la aplicabilidad del proyecto desde el punto de vista del desarrollo y la implantación de soluciones industriales. La posibilidad de simular virtualmente el robot, los sistemas de transporte, la cámara y el resto de los elementos de la célula permite realizar pruebas y modificaciones sin necesidad de disponer inicialmente de todos los componentes físicos. De esta manera, es posible analizar el comportamiento de la instalación, estudiar los movimientos del robot, comprobar posibles conflictos y evaluar los tiempos de funcionamiento antes de realizar una implementación física.

Además, el modelo desarrollado puede servir como base para una futura puesta en marcha de la célula en una instalación real. La programación realizada mediante la API de Python de RoboDK permite establecer una comunicación entre los diferentes elementos del sistema y facilita la transferencia de la lógica desarrollada al entorno físico. Antes de esta implementación sería necesario realizar las correspondientes adaptaciones y validaciones relacionadas con los equipos reales, las condiciones de iluminación, la calibración de la cámara, las comunicaciones y los sistemas de seguridad, pero el desarrollo realizado permite disponer de una primera validación de la arquitectura y del funcionamiento general de la célula.

Finalmente, la solución propuesta puede ser aplicable a diferentes sectores industriales en los que se requiera la inspección visual y clasificación automática de productos, especialmente en procesos caracterizados por tareas repetitivas y que tengan una necesidad de inspección de calidad. La combinación de robótica, visión artificial, sistemas de transporte y simulación permite desarrollar soluciones orientadas a mejorar la productividad, la repetibilidad y la calidad del proceso, al mismo tiempo que proporciona una metodología que puede ser adaptada a futuras aplicaciones de automatización industrial.

1.4. Descripción y funcionamiento del sistema automatizado

La célula desarrollada en este proyecto tiene como finalidad automatizar el proceso de inspección y clasificación de diferentes productos mediante la integración de un sistema de visión artificial, un robot colaborativo y diferentes sistemas de transporte. La instalación se ha planteado como una representación de una célula industrial real, en la que los productos llegan de forma automática a una zona de inspección, son analizados mediante una cámara y, en función del resultado obtenido, son clasificados como productos correctos o defectuosos. Todo el conjunto se encuentra coordinado de manera que los diferentes elementos de la instalación puedan trabajar de forma sincronizada y autónoma.

La célula está compuesta principalmente por un robot colaborativo Universal Robots UR5e, una cámara OnRobot Eyes instalada en el efector final del robot, una cinta transportadora de entrada, una cinta rotatoria y dos cintas transportadoras de salida. A estos elementos se añaden los diferentes elementos necesarios para el funcionamiento

de la instalación de forma segura, como la estructura de protección de la célula. La distribución de estos componentes se ha realizado buscando un equilibrio entre el espacio disponible, el alcance del robot y la accesibilidad a las diferentes zonas de trabajo.

El funcionamiento comienza con la llegada de los productos a través de la cinta transportadora de entrada. La cinta desplaza los productos hacia la zona destinada a la inspección, donde se produce la interacción entre el sistema de transporte, la cámara y el robot. Cuando una pieza alcanza la zona de inspección, el objeto es movido por la cinta rotatoria, que primero sitúa la pieza en el centro de esta, donde apunta la cámara, y dependiendo del objeto que sea, el robot se mueve en coordinación con la cinta rotatoria para realizar las distintas capturas necesarias. Una vez realizadas las capturas, las imágenes obtenidas son procesadas por el sistema de visión artificial. Este proceso permite analizar las características de la pieza y determinar si cumple con los criterios de calidad establecidos para la aplicación.

Una vez finalizada la inspección, el resultado obtenido es comunicado al resto de elementos de la célula. En función de si la pieza ha sido identificada como correcta o defectuosa, el sistema determina la orientación que debe tomar la cinta rotatoria y pasa el objeto ya escaneado a la cinta correspondiente, la célula dispone de dos líneas de salida diferenciadas para separar los productos que cumplen los criterios de calidad de aquellos que presentan algún defecto.

2. Estado del arte y antecedentes

2.1. Automatización industrial y células robotizadas

La automatización industrial constituye uno de los principales elementos de evolución de los procesos productivos actuales. La incorporación de sistemas automáticos permite realizar determinadas operaciones de forma repetitiva, precisa y controlada, reduciendo la intervención manual y mejorando aspectos como la productividad, la calidad y la repetibilidad del proceso. Dentro de este contexto, la robótica industrial se ha consolidado como una de las principales tecnologías utilizadas para automatizar operaciones de manipulación, montaje, transporte, soldadura e inspección.

Los robots industriales permiten ejecutar movimientos programados con una elevada precisión y repetir una misma operación durante largos periodos de tiempo. Además, su capacidad de programación permite adaptar sus movimientos a diferentes tareas y referencias de producto, proporcionando una mayor flexibilidad frente a determinados sistemas automatizados de funcionamiento exclusivamente mecánico. Según los datos publicados por la Federación Internacional de Robótica (IFR) en su informe World Robotics 2025, en 2024 se instalaron 542.000 robots industriales en todo el mundo, lo

que supone el cuarto año consecutivo con más de 500.000 unidades instaladas anualmente [1].

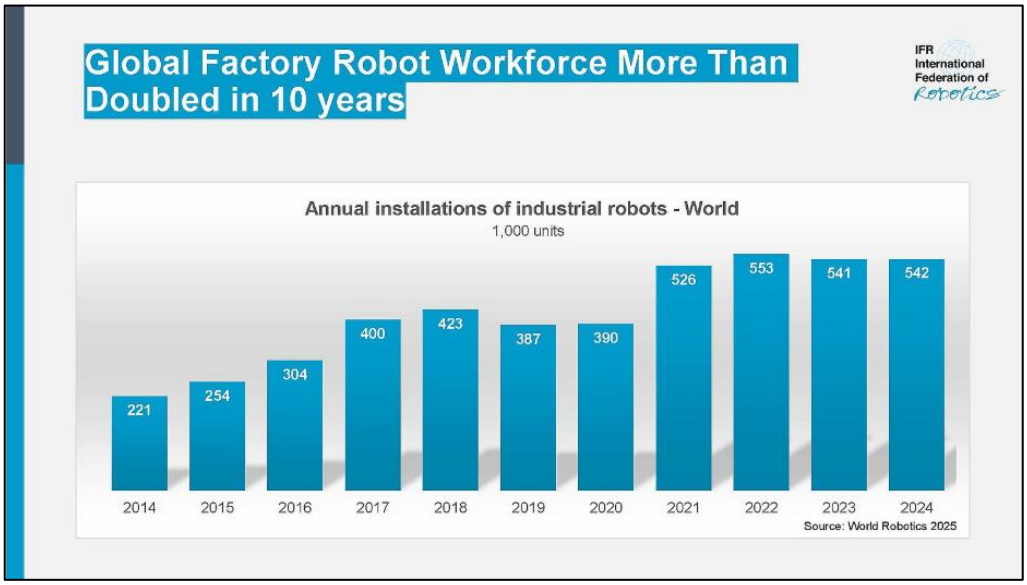


Figura 1: Instalaciones anuales de robots industriales entre 2014 y 2024. Obtenida de [1]

Sin embargo, un robot industrial normalmente forma parte de un sistema más amplio. Las células robotizadas integran diferentes elementos, como sistemas de transporte, sensores, herramientas de manipulación, dispositivos de seguridad y sistemas de control, que deben trabajar de forma coordinada para realizar la tarea prevista. Por ello, el diseño de una célula requiere considerar no solo las características del robot, sino también su interacción con el resto de los elementos y la distribución física de la instalación.

La evolución de la robótica industrial ha favorecido además la integración de los robots con otros sistemas de adquisición y procesamiento de información. La incorporación de sensores y sistemas de visión artificial permite que el robot pueda recibir información sobre el entorno y modificar su comportamiento en función del estado del proceso. Esto amplía las posibilidades de automatización y permite desarrollar células capaces de realizar tareas más flexibles, como la inspección y clasificación automática de productos.

En este contexto, las células robotizadas constituyen una solución ampliamente utilizada para automatizar procesos industriales en los que se requiere combinar manipulación, transporte y toma de decisiones. La integración de estos elementos constituye la base de las células de inspección automatizada.

2.2. Robótica colaborativa en procesos industriales

La evolución de la robótica industrial ha dado lugar al desarrollo de los robots colaborativos, también conocidos como *cobots*. El término *cobot* hace referencia a una categoría de robots pensada para trabajar en contacto directo con personas dentro de un

mismo espacio físico. Esta forma de automatización supone un cambio importante respecto al modelo clásico, en el que el robot se concebía como una máquina aislada que ejecutaba tareas repetitivas sin interacción directa con el operario [2]. La escasez de mano de obra cualificada, el aumento de los costes laborales y la necesidad de adaptar las líneas de producción a series cada vez más cortas y variadas han favorecido el crecimiento de este modelo de automatización, representando una proporción creciente del total de instalaciones de robots industriales a nivel mundial [1]

Lo que distingue a un *cobot* de un robot industrial es la inclusión de una serie de funciones de seguridad integradas: limitación de fuerza y potencia, detección de colisiones y parada inmediata ante un contacto no previsto. Estas características permiten reducir, e incluso eliminar en ciertos casos, la necesidad de recintos de seguridad físicos. No obstante, una tarea puede no ser segura completamente aun siendo realizada por un *cobot*, dependiendo del resto de elementos que influyan en esta. Por este motivo, la integración de un robot colaborativo requiere realizar una evaluación de riesgos y establecer las medidas de seguridad necesarias para cada aplicación, teniendo en cuenta también la herramienta, la pieza manipulada y el entorno de trabajo [2].

Además gracias a interfaces más accesibles que las de los sistemas industriales tradicionales, la programación de estos es mucho más accesible, frecuentemente realizada mediante el guiado manual o en entornos gráficos. Esta accesibilidad ha facilitado la incorporación de la robótica colaborativa en empresas de tamaño medio y pequeño

Actualmente, los *cobots* se emplean en tareas como el ensamblaje, el pick and place, el paletizado o la inspección de calidad, favorecidos por su capacidad de integración con sistemas de visión artificial y su flexibilidad frente a cambios de producto. No obstante, sus restricciones de velocidad y carga los hacen menos adecuados para aplicaciones de producción masiva a alta velocidad, terreno en el que los robots industriales convencionales siguen siendo más eficientes. La elección entre ambos debe hacerse teniendo en cuenta factores como el volumen de producción, la interacción con el operario, el espacio disponible y requisitos de seguridad.

2.3. Sistemas de visión artificial para inspección industrial

La visión artificial constituye en la actualidad una de las tecnologías más extendidas dentro de los procesos de control de calidad industrial. Un sistema de visión artificial está formado por un conjunto de elementos que trabajan de forma conjunta: un sistema de iluminación adecuado a la aplicación, uno o varios objetivos ópticos, una cámara encargada de la captura de la imagen, una interfaz de comunicación entre la cámara y el equipo de procesamiento, y un software especializado que analiza la imagen adquirida y determina si la pieza cumple los criterios establecidos [3].

De entre todos estos elementos, la iluminación desempeña un papel crítico en el resultado final de la inspección. Una configuración lumínica inadecuada puede generar

sombras, reflejos o variaciones de contraste que dificulten la posterior extracción de características, mientras que una iluminación bien diseñada permite resaltar los defectos que se pretenden detectar y reducir la influencia de las condiciones ambientales [3].

Tradicionalmente, los sistemas de visión artificial se han basado en técnicas de procesamiento como la segmentación, la extracción de bordes o la correlación por patrones, que ofrecen resultados robustos y predecibles cuando las condiciones de captura son estables y los defectos a detectar están bien definidos [3]. Estos métodos, aunque siguen empleándose ampliamente en la industria por su transparencia y bajo coste computacional, presentan limitaciones frente a piezas de geometría compleja, materiales no rígidos o defectos con una gran variabilidad de apariencia.

En los últimos años, la incorporación de técnicas de aprendizaje automático y aprendizaje profundo ha ampliado notablemente las capacidades de los sistemas de inspección visual, permitiendo abordar tareas de clasificación, detección y segmentación de defectos a partir de conjuntos de datos de entrenamiento, sin necesidad de definir manualmente reglas o umbrales específicos para cada tipo de defecto [3].

Por otro lado, en cuanto a la naturaleza de la información capturada, los sistemas de visión pueden clasificarse en sistemas de visión 2D, que analizan la imagen proyectada de la pieza, y sistemas de visión 3D, que además incorporan información de profundidad mediante técnicas como la luz estructurada, la triangulación láser, la estereovisión o los sistemas de tiempo de vuelo. La elección entre una u otra tecnología depende de factores como la precisión requerida, la distancia de trabajo, el tipo de defecto a detectar y las condiciones del entorno de producción [3].

2.4. Integración de visión artificial y robótica

La combinación de sistemas de visión artificial con robots constituye uno de los elementos clave de las células robotizadas de inspección. Esta integración permite que el robot adapte su comportamiento en función de la información visual obtenida en cada momento, ya sea para localizar una pieza, guiar una operación de manipulación o posicionarse adecuadamente para la captura de imágenes durante una inspección [4].

Para que esta integración funcione correctamente, es necesario establecer una relación precisa entre el sistema de coordenadas de la cámara y el del robot, de forma que la información extraída de la imagen pueda traducirse en movimientos reales. Este proceso, conocido como calibración mano-ojo (*hand-eye calibration*), resulta crítico en cualquier aplicación de robótica guiada por visión, ya que la precisión final del sistema depende en gran medida de la exactitud con la que se determine dicha transformación [4].

Existen dos configuraciones principales según la posición de la cámara respecto al robot. En la disposición *eye-in-hand*, la cámara se monta sobre el efector final y se desplaza junto con el robot, lo que resulta especialmente útil en tareas de inspección al permitir adquirir imágenes desde distintas posiciones y orientaciones según la geometría de la pieza. En la disposición *eye-to-hand*, la cámara permanece fija en el entorno

mientras el robot se mueve dentro de su campo de visión, lo cual es más común en aplicaciones de pick and place en las que se detecten constantemente objetos para conocer su posición [5].

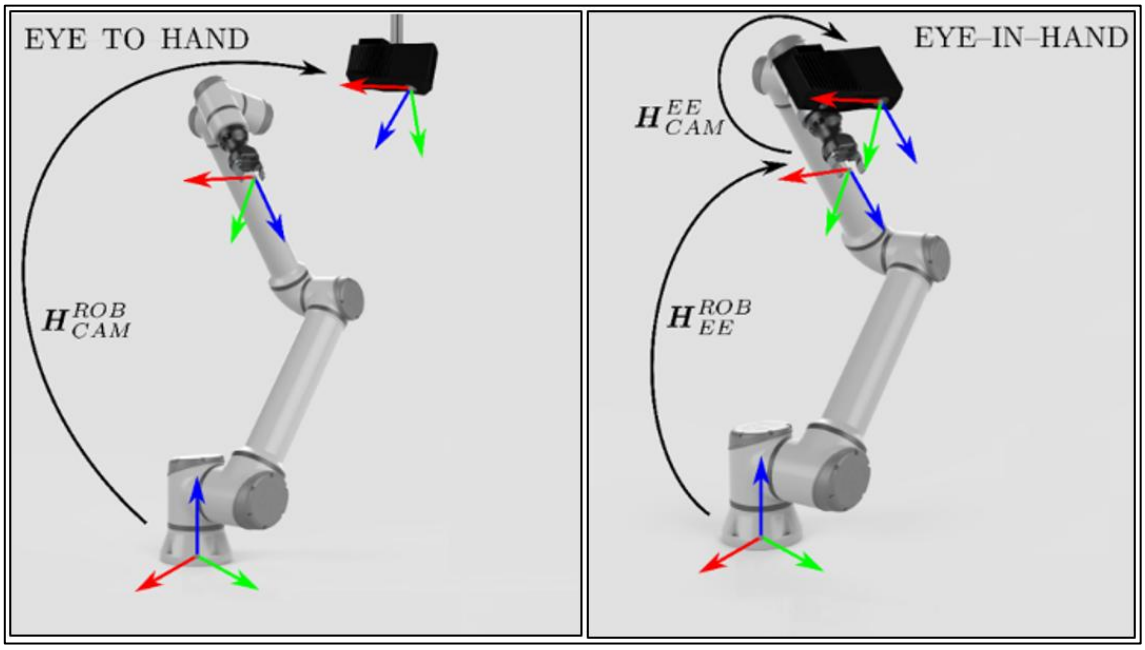


Figura 2: Representación gráfica de la diferencia entre las configuraciones eye-to-hand y eye-in-hand. Obtenida de [5]

En aplicaciones de inspección visual guiada por robot como la desarrollada en este proyecto, la configuración *eye-in-hand* resulta particularmente adecuada, ya que aprovecha los grados de libertad del robot para adaptar la posición de la cámara a cada pieza, proporcionando una mucho mayor flexibilidad que en un sistema de visión fijo con un único punto de vista. Esta capacidad es especialmente relevante en la inspección de piezas de geometría compleja o tridimensional, donde puede ser necesario capturar varias imágenes desde distintos ángulos.

Por último, la correcta integración entre robot y sistema de visión no depende únicamente de la calibración geométrica, sino también de la sincronización temporal del proceso: el robot debe conocer cuándo detenerse o desplazarse para permitir la captura de la imagen, y el sistema de visión debe comunicar de forma fiable el resultado del análisis para que el resto de la estación actúe con la información obtenida.

2.5. Técnicas de procesamiento de imágenes aplicadas a la inspección

La capacidad de un sistema de visión artificial para detectar defectos depende, en gran medida, del conjunto de técnicas de procesamiento que se le apliquen a dicha imagen. Estas técnicas suelen organizarse en dos niveles: un procesamiento de bajo nivel, encargado de preparar la imagen para el siguiente nivel (eliminación de ruido, ajuste de contraste, filtrado), y un procesamiento de alto nivel, encargado de interpretar la información presente en la misma para determinar si la pieza cumple los criterios de calidad establecidos [6].

Dentro de las técnicas de procesamiento, la segmentación ocupa un lugar central, ya que permite aislar la región de la imagen correspondiente a la pieza o a la parte de esta que requiera ser inspeccionada, separándola del fondo y del resto de elementos presentes en la imagen. Una vez realizada esta segmentación resulta posible aplicar técnicas de extracción de características, orientadas a identificar atributos concretos como bordes, texturas o formas, que servirán posteriormente para comparar la pieza con un patrón de referencia [7].

La detección de bordes constituye una de las operaciones más habituales en este tipo de sistemas, al permitir localizar los cambios bruscos de intensidad en la imagen, que suelen corresponder a los contornos reales de los objetos. Algunos operadores como los de Sobel o Prewitt son utilizados para calcular el gradiente de intensidad en las cercanías de cada píxel, consiguiendo de esta forma detectar donde se presenta un borde en una imagen y la orientación de este [6].

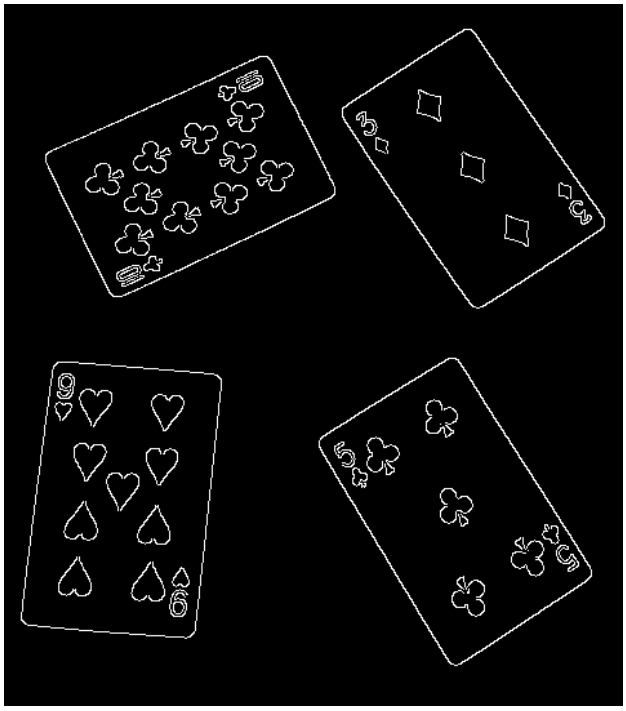


Figura 3: Ejemplo de detección de bordes en imagen. Obtenida de [8]

Junto a estas técnicas, el emparejamiento por patrones representa otro de los enfoques más utilizados en la inspección industrial. En este método se compara la imagen capturada con una imagen modelo de referencia previamente definida, evaluando su grado de similitud en distintas posiciones de búsqueda; su principal ventaja se da en su relativa facilidad de uso, ya que requiere poco conocimiento técnico para su configuración, bastando con proporcionar imágenes de referencia durante una fase de entrenamiento. Sin embargo, este enfoque presenta limitaciones relevantes en las zonas de borde de la pieza, donde las variaciones normales de los objetos pueden confundirse con defectos reales, siendo esta una de sus principales debilidades para aplicaciones de inspección muy exigentes [9].

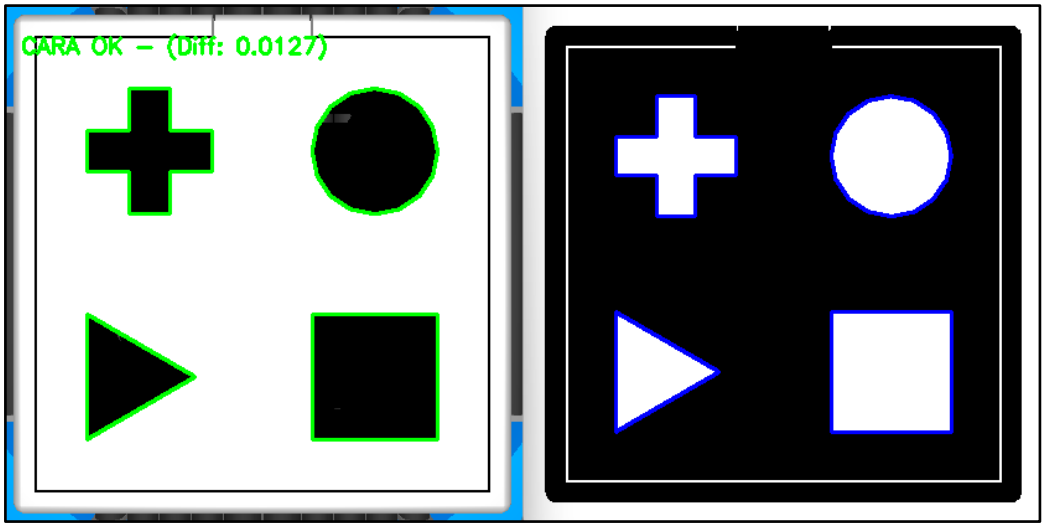


Figura 4: Ejemplo de emparejamiento por patrones en la célula, a la izquierda la imagen capturada del objeto a escanear y a la derecha la imagen de referencia

En la práctica, los sistemas de inspección industrial rara vez emplean una única técnica de forma aislada, sino que combinan varias de ellas como segmentación, detección de bordes y emparejamiento por patrones, en función del tipo de defecto a detectar y de las características de la pieza, con el objetivo de optimizar la precisión de los resultados y la velocidad del proceso de clasificación [10].

2.6. Simulación y gemelos digitales en robótica industrial

2.6.1. El concepto de gemelo digital

El término gemelo digital hace referencia a una réplica virtual de un sistema físico, en el que se ve reflejado el comportamiento de este, con el fin de analizarlo, probarlo o predecir su funcionamiento sin la necesidad de realizar cambios o parar el funcionamiento del equipo real. En el ámbito de la robótica industrial, el gemelo digital, no permite modelar únicamente el robot, sino que también permite la representación del resto de elementos que componen la célula robotizada, como las herramientas, los utilajes, cintas transportadoras, sensores y piezas, es decir, todo aquello que intervenga o sea relevante en el proceso automatizado [11].

Esta tecnología ha experimentado un gran crecimiento gracias al desarrollo del paradigma de la Industria 4.0, ya que en este los gemelos digitales se han convertido en una herramienta muy importante tanto para la monitorización, como para la simulación y la optimización de sistemas productivos. Gracias al empleo de esta tecnología se pueden lograr mejoras en aspectos como el mantenimiento predictivo, la mejora de la eficiencia del proceso y una mayor facilidad a la hora del diseño de nuevas instalaciones o de la automatización de las existentes [11].

El gemelo digital puede ser empleado en diferentes fases de un proyecto de automatización. En la fase de diseño, permite comprobar que todos los componentes

son adecuados para la tarea y que no generan interferencias ni problemas entre ellos antes de realizar el montaje físico de ninguno de estos. Durante la fase de programación, permite que la programación del robot sea realizada incluso antes de que se disponga del equipo físicamente, siguiendo el concepto del *offline programming* (OLP). Por último, una vez el proyecto está instalado y en funcionamiento, el gemelo digital puede seguir siendo empleado a la hora de introducir nuevas modificaciones, resolver problemas o para monitorizar el funcionamiento del sistema en tiempo real [12].

Sin embargo, gran parte de las herramientas de simulación actuales presentan limitaciones a la hora de representar de forma correcta el comportamiento de ciertos componentes, especialmente cuando se trata de señales procedentes de sensores externos, lo que supone un reto por cumplir para conseguir una representación completamente real de un sistema físico en su réplica virtual [11].

2.6.2. RoboDK como herramienta de simulación

Entre las distintas herramientas de simulación y offline programming, podemos encontrar dos grandes grupos. Por un lado se encuentran las soluciones específicas de fabricante, como ABB RobotStudio, FANUC ROBOGUIDE o KUKA.Sim. Estas suelen ofrecer una integración muy precisa con los robots del propio fabricante, ya que están diseñadas específicamente para estos, pero su uso queda normalmente limitado a robots de su misma marca, lo que supone una limitación en entornos en los que colaboran robots de diferentes proveedores.

Por otro lado se encuentran las soluciones universales o multiproveedor, como RoboDK, Robotmaster o Visual Components. Este tipo de soluciones ofrecen compatibilidad con robots de múltiples fabricantes, lo que las hace una opción especialmente adecuada para instalaciones en las que se combinan robots de diferentes proveedores, ya que no requiere el aprendizaje de un software específico por cada uno de los fabricantes que se empleen.

En el presente proyecto se ha optado por el uso de la herramienta RoboDK, un software de simulación y programación de robots industriales, que permite la simulación mediante modelos tridimensionales de células robotizadas completas. Estas simulaciones son compatibles con más de 80 fabricantes de robots distintos, lo que facilita su empleo en una gran variedad de aplicaciones y configuraciones [13].

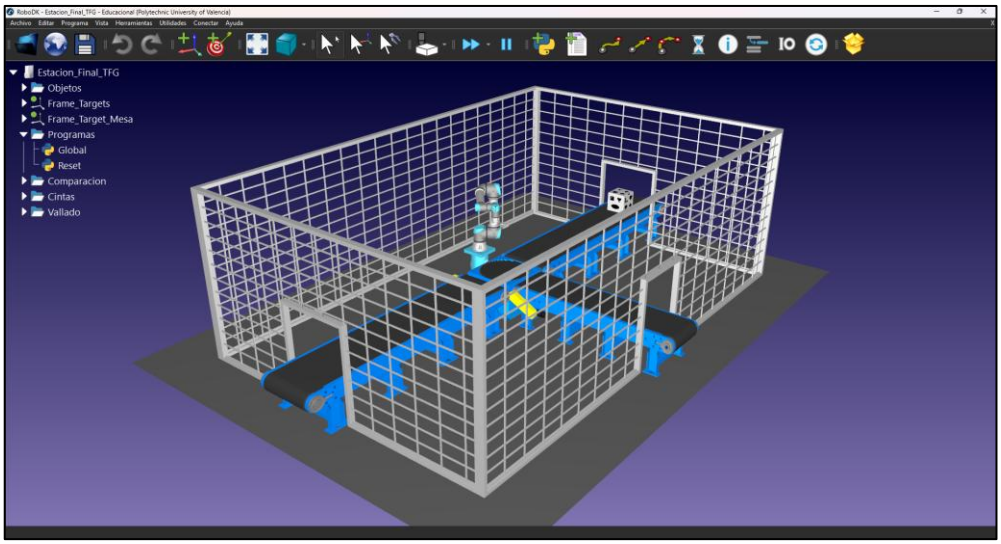


Figura 5: Interfaz de RoboDK en el presente proyecto

Esta herramienta permite definir el layout del entorno de trabajo del robot, programar los movimientos de este y simular el funcionamiento de la planta antes de que sea implementada de forma física, pudiendo así detectar posibles problemas como singularidades en la cinemática del robot, obstáculos con los límites de las articulaciones o colisiones con otros elementos [13]. Además, RoboDK permite la generación de código específico para los robots utilizados en el lenguaje de programación propio de cada fabricante, utilizando las trayectorias definidas en el entorno de simulación. Este código puede ser exportado y transferido al controlador del robot físico, consiguiendo así ejecutar el programa que ya ha sido comprobado en el entorno simulado. De esta forma, se reduce el tiempo dedicado a la programación manual del robot una vez instalado. RoboDK también permite la programación en Python mediante su API, lo que aporta una mayor flexibilidad con respecto a los softwares específicos de los fabricantes [13].

2.6.3. Uso de los gemelos digitales en la robótica industrial

El uso de gemelos digitales en el diseño y la instalación de células robotizadas aporta ciertas ventajas con respecto a los métodos de programación tradicionales, basados en la operación de forma directa sobre el robot real. En primer lugar, permite reducir los tiempos de puesta en marcha de una instalación, al poder realizar el desarrollo de esta sin esperar a la llegada de los dispositivos necesarios ni detener la producción, siendo esta última especialmente importante en instalaciones en las que el robot a programar ya se encuentre integrado en otra línea de producción paralela [14].

En segundo lugar, el uso de un gemelo digital hace que el proceso de desarrollo sea más seguro, ya que permite detectar de forma anticipada posibles colisiones o interferencias, eliminando así riesgos producidos por la depuración de programas directamente sobre el sistema físico [12]. Esta característica es especialmente útil en el ámbito de la

inspección visual, en la que el robot debe alcanzar varias posiciones dependiendo del objeto, que pueden llegar a ser muy complejas, por lo que de esta forma se puede comprobar que todas estas son viables y alcanzables por el mismo.

Por último, los gemelos digitales también permiten que la fase de mantenimiento de una instalación y la posible evolución de esta se realicen de forma mucho más fácil, ya que se pueden probar las futuras evoluciones en la simulación antes de ser trasladadas al entorno real, detectando así posibles fallos pasados por alto y evitando paradas que pudieran derivar de estos [14].

2.7. Tecnologías y soluciones comerciales existentes

Además de sistemas de visión artificial hechos a medida por encargo, en el mercado actual se pueden encontrar sistemas de visión artificial listos para ser integrados en un proceso industrial con una relativamente baja necesidad de llevar a cabo configuraciones específicas. Esos sistemas se pueden clasificar en dos grandes categorías: cámaras inteligentes, que integran en un mismo dispositivo el sistema de adquisición de imagen, el procesador y el software de análisis de las imágenes; y los sistemas basados en controlador, en los que una cámara o una serie de cámaras se conectan a una unidad de procesamiento independiente con una mayor capacidad de procesamiento en la que se encuentra instalado un software de procesamiento [15].

En cuanto al primer grupo, se pueden encontrar soluciones como la serie Cognex In-Sight, usada normalmente en aplicaciones como la inspección de componentes electrónicos o el control de ensamblajes. Estas integraciones se realizan normalmente mediante una integración directa con PLC para automatizar por completo la decisión de aceptar o rechazar una pieza [15]. Otra solución perteneciente al primer grupo es la serie FH de Omron, que dispone de un algoritmo de búsqueda *Shape Search III*, que consigue mantener una buena fiabilidad en la identificación de piezas aun cuando las condiciones de captura de la imagen no son óptimas. Además, también proporciona funciones de inspección mediante inteligencia artificial, lo cual la hace capaz de detectar defectos no definidos previamente [16].

A la hora de construir un sistema basado en controlador, hay diferentes opciones para un software de procesamiento, estos pueden ser comerciales o librerías de código abierto. Entre las opciones comerciales se encuentra MVTec Halcon, un software ampliamente utilizado en aplicaciones industriales ya que incorpora la combinación de técnicas clásicas de procesamiento con funciones de visión 3D y deep learning [17]. Además de soluciones comerciales, también se puede crear un sistema de visión mediante el uso de librerías de código abierto, como es el caso de OpenCV. Siendo esta una librería gratuita ampliamente utilizada, con un extenso conjunto de algoritmos de procesamiento, además de incorporar modelos de deep learning mediante métodos como ONNX [18]. El uso de este método ofrece una mayor libertad de diseño y conlleva normalmente un gasto inferior, pero también añade tareas al integrarlo que un

modelo comercial integrado ya traerían resueltas de fábrica, como el diseño de la iluminación o la integración con el entorno.

La elección entre una de estas alternativas depende de las necesidades de cada aplicación, siendo los sistemas integrados más comunes en aplicaciones sencillas y estandarizadas, las plataformas comerciales más comunes en aplicaciones más complejas y que requieran un mayor grado de adaptación, y el uso de librerías de código abierto en proyectos de investigación y académicos donde se requiere un control total sobre el Sistema.

3. Metodología

3.1. Elección de los sistemas para el proyecto

3.1.1. Elección del robot

La elección del robot se llevó a cabo de forma cuidadosa, ya que es uno de los sistemas más importantes dentro de la celda de automatización, por ello se tuvieron en cuenta numerosos factores. En primer lugar, la tarea requiere de un nivel máximo de carga del robot muy bajo, ya que el robot únicamente tendrá que contar con la cámara fijada en su efector final, por lo que la carga máxima del robot no suponía un factor muy importante.

Además, el robot necesita una gran cantidad de grados de libertad, para poder realizar inspecciones en piezas de distintas formas y distintos grados de complejidad. Esto lleva a la elección de un robot antropomórfico (Figura 6) con 6 grados de libertad, ya que permite alcanzar la mayor cantidad de posiciones y orientaciones. Esto presenta una ventaja ante otras configuraciones, como los robots delta, SCARA o cartesianos, normalmente utilizados en aplicaciones en las que prima más la velocidad o la capacidad de carga que la flexibilidad y el alcance del robot (comparación entre configuraciones en figura 6).

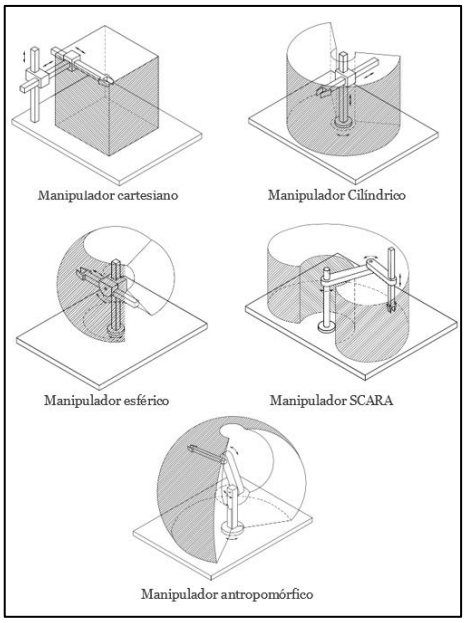


Figura 6: Comparación entre distintas configuraciones de robot. Elaboración propia a partir de [19]

Teniendo esto en cuenta, se acabó eligiendo el cobot UR5e (Figura 7), ya que este se trata de un robot antropomórfico con una gran posabilidad y que cuenta con un alcance de 850mm y un rango de rotación de ± 360 ° en sus articulaciones, suficiente para llevar a cabo la tarea propuesta [20]. Además, por tratarse de un cobot, este presenta ventajas ante un robot industrial convencional, como su mayor ligereza, lo que lo hace más fácil de transportar en caso de que hubiera que realizar cambios en la instalación, o su capacidad de detección de colisiones. Otra ventaja con la que cuenta este robot es la inclusión de una pantalla de 12 pulgadas, en la que se puede programar el robot mediante una interfaz gráfica, lo que facilita la formación de los operarios en caso de que fuera necesario realizar ajustes en el programa.

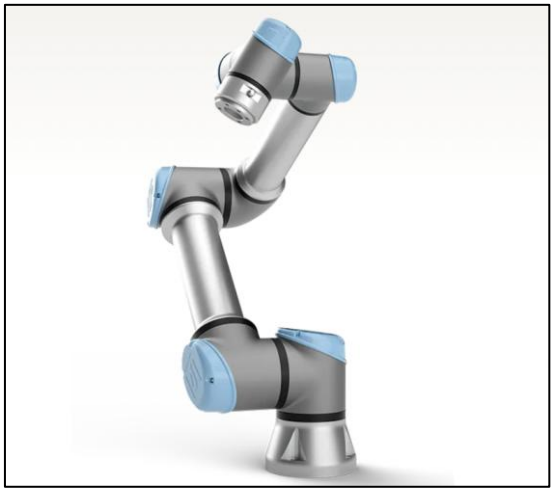


Figura 7: Imagen del cobot UR5e. Obtenida de [21]

3.1.2. Elección de la herramienta de simulación

El entorno de simulación elegido ha sido RoboDK (Figura 5), debido a factores como la gran flexibilidad que proporciona, la capacidad de exportar programas al robot en caso de que se instalara físicamente el proyecto, y su amplia librería disponible en la que se pueden encontrar una gran cantidad tanto de elementos robóticos, como mecanismos de transporte y herramientas.

Otra de las ventajas que presenta RoboDK es la compatibilidad con herramientas de CAD y la posibilidad de importar objetos propios a partir de modelos en formatos CAD. Esto ha permitido la importación de todos los objetos a inspeccionar y elementos auxiliares como el vallado o el pedestal sobre el que se monta el robot. Además, RoboDK permite la creación de nuevos mecanismos a parte de los ya existentes, lo cual ha sido especialmente útil a la hora de crear la mesa rotatoria sobre la que se encuentra montada la cinta que se encarga de la clasificación de los objetos una vez inspeccionados (Figura 8), que, al necesitar estar hecha a medida, era imposible de simular sin una herramienta de creación de mecanismos como la que proporciona RoboDK.

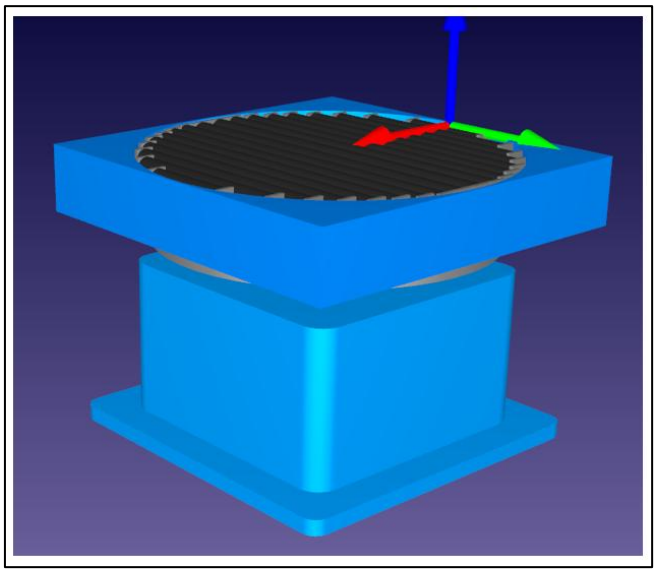


Figura 8: Imagen de la mesa rotatoria

Por último, RoboDK también permite la simulación de cámaras, lo cual era un requisito indispensable para la realización de este proyecto, en el que la visión artificial tiene un papel clave para el funcionamiento de este. Por lo que la posibilidad de realizar la programación del procesamiento de las imágenes en Python, haciendo uso de la librería OpenCV, también representa una gran ventaja a la hora de elegir este entorno.

3.1.3. Elección de la cámara

La cámara elegida para este proyecto ha sido la OnRobot Eyes, que cuenta con un sistema de visión 2.5D, y que está diseñada específicamente para su integración en

robos colaborativos. Esta cámara cuenta con la certificación del programa UR+, por lo que está probado y diseñado específicamente para su funcionamiento con robots UR. Además, cuenta con integración nativa con Universal Robots, ya que dispone de una URCap propia (un plugin de software que se instala en la Tablet desde la que se programa el robot (*Polyscope*)) gracias a la cual se puede manejar la cámara desde el mismo *Polyscope* que se controla la programación del robot.

Además, también presenta flexibilidad para operaciones futuras en las que se requiera el uso de otras herramientas además de la cámara acopladas al robot. Esto se consigue gracias a su sistema de conexión *OnRobot Quick Changer*, que permite la conexión de una pinza de OnRobot en el mismo acople sin la necesidad de instalar ningún cableado adicional.

3.2. Diseño de las piezas a escanear

El primer paso a la hora de realizar el proyecto fue el diseño y modelado en 3D de las piezas a inspeccionar dentro de la célula. El diseño y modelado de estas, incluyendo versiones con distintos defectos de estas, y su posterior exportación fueron realizados mediante la aplicación *Inventor* de Autodesk. Estas piezas fueron diseñadas para exponer el funcionamiento de la célula de inspección en distintos ámbitos y en objetos de distintas formas.

La primera pieza se trata de una plantilla de zapato simplificada, esta fue elegida como una pieza plana que pudiera demostrar el funcionamiento del sistema con formas irregulares y sin posibilidad de usar profundidad, lo que obliga a que la inspección sea realizada mediante el uso de imágenes y la comparación con imágenes maestras. Tanto plantillas del pie izquierdo como el derecho fueron diseñadas, cada una con sus respectivos modelos con errores en el proceso de fabricación (Figura 9).

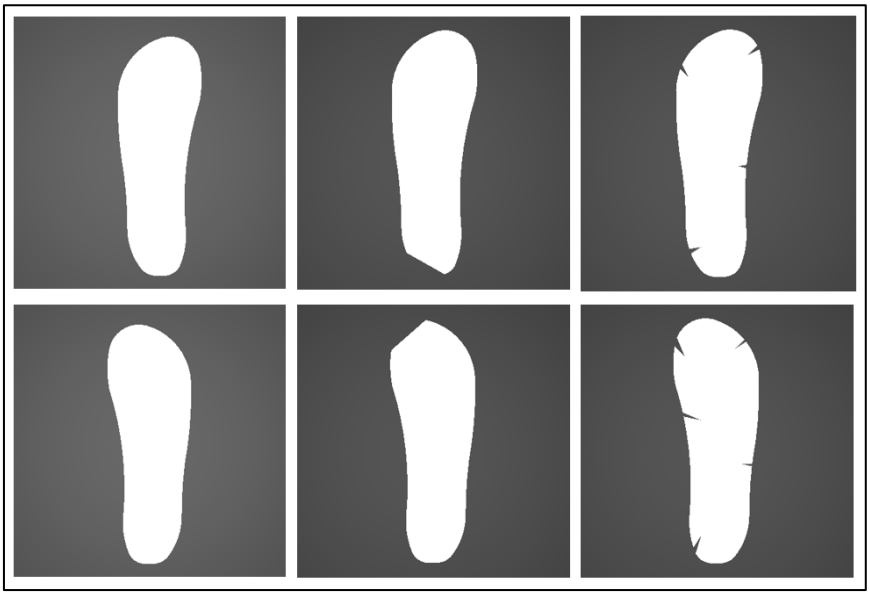


Figura 9: Imágenes de las distintas variantes de plantillas dentro de RoboDK

La segunda pieza diseñada se trata de un juguete para niños, normalmente de madera, en el que se encajan piezas de distintas formas en agujeros situados en todas las caras del cubo menos la inferior (Figura 10). Esta pieza fue elegida para mostrar el funcionamiento de la célula en situaciones en las que se han de reconocer distintas formas en una misma cara, como en caso de una pieza que requiera tener una serie de taladros en lugares específicos, además de mostrar el funcionamiento de la estación ante objetos con una forma tridimensional, en las que el posicionamiento y la flexibilidad en las posturas del robot en combinación con la mesa rotatoria es fundamental para lograr un funcionamiento óptimo. Cada una de las caras cuenta con un patrón y unas formas diferentes para probar el funcionamiento en condiciones diferentes, y se han diseñado 5 objetos diferentes con cada uno un error en una de las caras, siguiendo con la comprobación del funcionamiento en distintos casos.

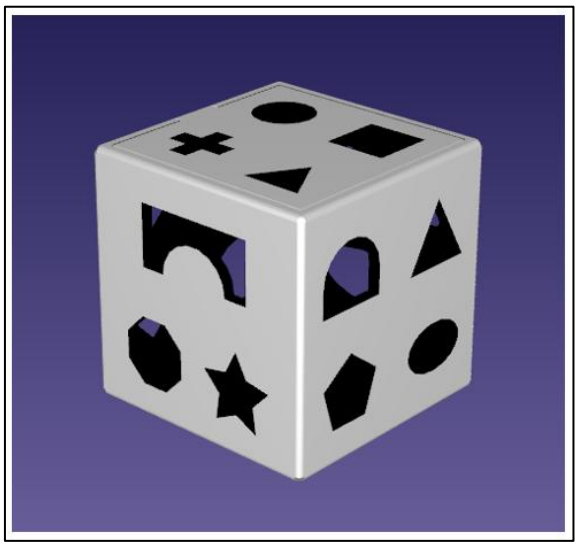


Figura 10: Imagen del cubo en RoboDK

Por último, la tercera pieza a inspeccionar se trata de una taza de desayuno. Este objeto se seleccionó para comprobar y demostrar el correcto funcionamiento de la solución tanto para el reconocimiento en superficies curvas, como para el reconocimiento de errores en la impresión de textos o logotipos. Este objeto también cumple el factor de ser tridimensional, por lo que también hace uso de la flexibilidad que proporcionan la combinación de la mesa rotatoria con el robot. En este caso, los errores introducidos dentro del diseño de la pieza no han sido en el objeto como tal, si no en la impresión del logotipo con el que cuenta en el centro de esta, aprovechando el reconocimiento de patrones y de formas complejas desarrollado para los objetos anteriores (Figura 11).



Figura 11: Imagen de las distintas variantes de tazas dentro de RoboDK

3.3. Diseño del proceso y el layout

3.3.1. Diseño del proceso

El proceso puede ser dividido en una serie de fases que se repiten de forma constante en un sistema cíclico. Estas fases son:

1. **Llegada de productos:** los distintos productos producidos en la fábrica llegan a través de la primera cinta en orden aleatorio y con una orientación ya establecida previamente.
2. **Llegada a la mesa rotatoria:** mediante un sensor de barrera, se detecta cuándo llega el próximo objeto al final de la primera cinta, en ese momento se activa la cinta situada sobre la mesa rotatoria y avanza el objeto hasta que este está situado en su centro, donde se encontrará en la posición óptima para la captura de imágenes por el robot.
3. **Captura de imágenes del objeto:** dependiendo de la clase de objeto que se vaya a inspeccionar, la célula llevará a cabo un proceso distinto.
    a. Plantilla: En el caso de la plantilla, al tratarse de un elemento plano, únicamente se toma una imagen, por lo que el robot se sitúa de forma perpendicular al plano de la cinta y toma una única captura.
    b. Cubo: Para la captura de las distintas imágenes del cubo, el robot funciona en sincronía con la cinta y la mesa rotatoria. Primero, se toma una imagen de la cara superior, a continuación, el robot cambia de posición para apuntar a la cara frontal del cubo, y una vez está tomada la imagen frontal la mesa rotatoria girará en intervalos de 90 grados mientras el robot se mantiene inmóvil. Esto se hace así para evitar movimientos innecesarios por parte del robot, optimizando de esta forma el tiempo de escaneo.
    c. Taza: Al igual que con el cubo, para la captura de imágenes de la taza, el robot trabaja en conjunto con la cinta y mesa rotatoria. En este caso, la mesa rotatoria gira 90 grados en sentido antihorario, exponiendo el diseño de la taza hacia el robot, que se sitúa en paralelo a la cinta. Después, la cinta sobre la mesa rotatoria retrocede hasta que la taza está lo suficientemente cerca de la cámara como para que la imagen tomada tenga una cantidad de detalle suficiente.

4. **Procesamiento de imágenes:** Tras la captura de cada una de las imágenes, a estas se les aplica un programa de Python específico dependiendo del objeto a analizar. Esto es así ya que para cada uno de los objetos se han de definir unas tolerancias específicas para que funcionen de forma óptima. Sin embargo, el funcionamiento básico de estos programas es común ya que usan el mismo sistema de identificación.
5. **Clasificación de los productos:** Una vez se ha identificado si un producto es válido o defectuoso, la cinta rotatoria se encarga de enviarlo por la cinta pertinente. Esto lo realiza primero rotando hacia una de sus dos posiciones predeterminadas, para después avanzar la cinta hasta que uno de los dos sensores de barrera situados en cada una de ellas detecte que el objeto está preparado para su evacuación.
6. **Evacuación de los productos:** Cuando los objetos se encuentran en la cinta que les corresponde, estas avanzan una distancia predeterminada por cada objeto que llega, dejando lugar para la llegada de los próximos objetos. En la simulación, para evitar sobrecargas de memoria y limitaciones en los recursos de computación, los objetos son eliminados de la escena una vez salen de los límites del vallado de la célula.

3.3.2. Diseño del layout

El layout de esta estación cuenta con varios elementos:

Primero, una línea de alimentación por la que llegan los productos de forma aleatoria. Esta cinta es un modelo de RoboDK redimensionada para su uso específico en esta aplicación. Esta tiene 512 mm de ancho total (mientras que el mecanismo sobre el que se sitúan los objetos tan solo cuenta con 440 mm), 366.5 mm de alto y una longitud de 2330 mm. Esta cinta es la encargada de la entrada de todos los objetos para su posterior inspección.

Además de esta cinta, también se cuenta con otras dos, las encargadas de la salida de los productos, ya sea por el lado de los defectuosos o por el de los correctos. La cinta encargada de la evacuación de los productos correctos es igual que la encargada de la llegada de los productos, mientras que la cinta encargada de la evacuación de los productos defectuosos es diferente. Esta cuenta con 410 mm de ancho total (siendo 352 mm de ancho de la banda transportadora) y una longitud de 1630 mm, mientras que la altura se mantiene igual para permitir la comunicación con la mesa rotatoria.

Uno de los ejes centrales del funcionamiento de esta estación es la mesa rotatoria con una cinta montada sobre ella. Esta es la encargada de facilitar, y en algunas ocasiones hacer posible, la toma de imágenes de los objetos de parte del robot, además de ser la encargada de la posterior clasificación de los productos una vez identificados. La mesa rotatoria cuenta con una altura de 280 mm, y los lados del cuadrado de la base miden 440 mm, mientras que el disco rotatorio sobre el que se monta la cinta tiene un radio de 220 mm. La cinta montada sobre esta mesa cuenta con una estructura de protección

cuadrada de 440 mm de lado, dentro de la cual se encuentra el mecanismo de la cinta que rota de forma independiente a esta. La cinta abarca un radio de 237.5 mm, lo que le permite lograr el intercambio de objetos con las otras cintas evitando que estos caigan en la conexión entre las cintas.

Rodeando la zona de cintas, se encuentra un vallado industrial que cuenta con aperturas en los lugares por donde entran y salen las cintas de la estación. Este vallado sirve para aislar la célula y evitar que se produzcan accidentes por intervención humana o con otros procesos. Este vallado está hecho a medida mediante el uso de vallas estándar de protección industrial de 1400 mm de altura, y forman un rectángulo de 4420 mm x 2890 mm, lo cual constituye la huella final de la estación. Además, incluye 3 puertas de acceso para tareas de mantenimiento de los mecanismos.

El último elemento del layout es el robot UR5e, montado sobre un pedestal de forma cuadrada con lados de 250 mm y que se extiende hasta lograr una altura de 366.5 mm. El robot cuenta con un alcance de 850 mm, aunque su espacio de trabajo no lo abarca al completo en esta aplicación.

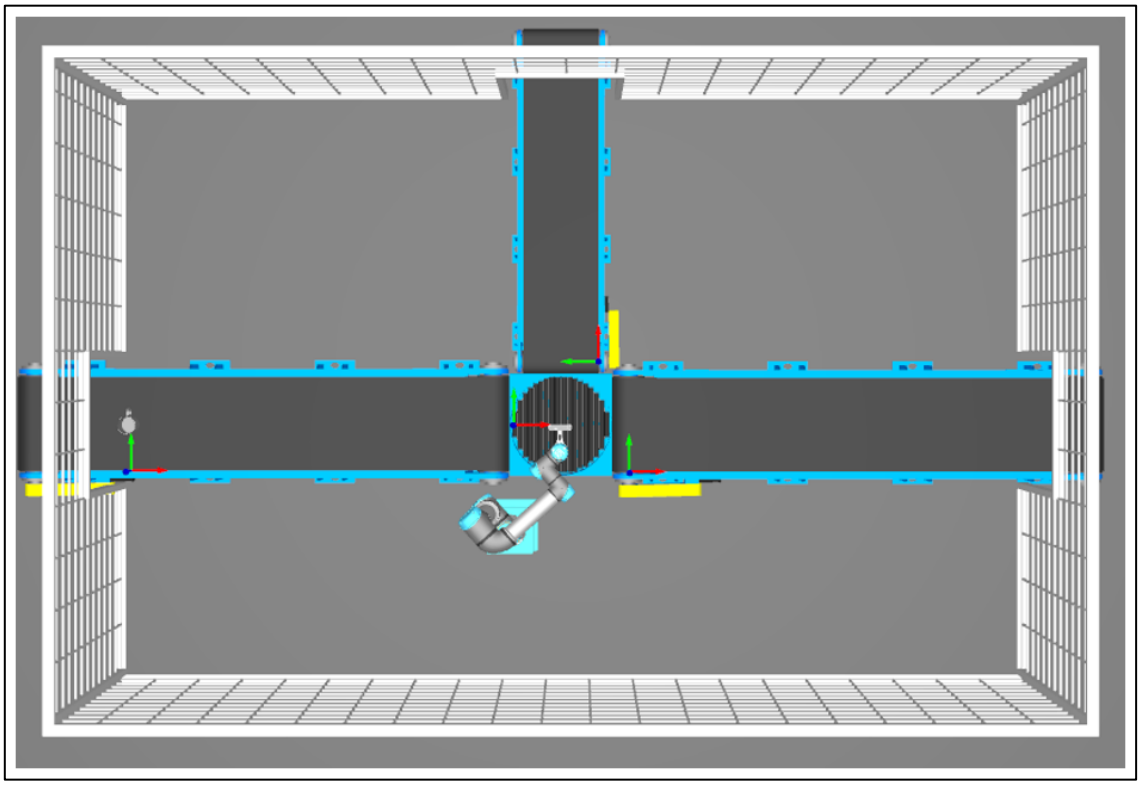


Figura 12: Captura del layout de la estación en RoboDK



Figura 13: Layout de la estación con nombres y cotas en milímetros

3.4. Estudio de seguridad industrial

3.4.1. Marco normativo

A la hora de realizar el estudio de seguridad para una estación de automatización de este tipo, las decisiones a tomar deben estar regidas en base al cumplimiento de una serie de normas establecidas. Las normas que aplican a este proyecto son:

- **ISO 12100:2010 – Seguridad de las máquinas. Principios generales para el diseño:** esta es la norma principal a partir de las que se desarrollan las demás. Esta norma no está centrada en las aplicaciones robóticas exclusivamente, sino que define la forma de evaluar y reducir los riesgos durante el uso de cualquier máquina. Esta norma establece que se deben delimitar los usos previstos y el mal uso previsible de una máquina, identificar todos los peligros, y evaluar el riesgo de cada uno de estos [22].
- **ISO 10218-1:2025 – Robots – Requisitos de seguridad – Parte 1: Robots industriales:** Esta norma trata al robot como un producto, es decir, trata los requisitos que el propio fabricante debe cumplir durante el diseño y la fabricación del robot. Esto comprende el diseño de elementos mecánicos para evitar riesgos, controles de seguridad, paradas de emergencia, limitaciones de velocidad y diferentes modos de funcionamiento [23].
- **ISO 10218-2:2025 – Robots – Requisitos de seguridad – Parte 2: Aplicaciones de robots industriales y células robóticas:** Esta norma es aplicable a proyectos en los cuales se integre un robot en una instalación. En ella

se describen los peligros básicos y las posibles situaciones de riesgo que se pueden dar en un sistema de estas características, además también establece los requisitos para poder eliminar esos riesgos de forma adecuada mediante elementos de seguridad como barreras físicas, cortinas de luz, alfombras de seguridad o escáneres de área [23].

- **ISO/TS 15066:2016 – Robots y sistemas robóticos – robots colaborativos:** En esta norma se amplía mucho mas acerca de los requisitos específicos que deben ser cumplidos por los robots colaborativos, especialmente aborda los límites de fuerza y presión que un robot puede ejercer cuando trabaja con humanos. Además este documento establece pautas para la evaluación de riesgos de una aplicación de robótica colaborativa. En ella se establecen los 4 modos de colaboración que se pueden dar cuando un cobot y un humano comparten el espacio de trabajo [23]:
    1. **Parada de seguridad supervisada:** el robot se detiene por completo siempre que el operario entre en su espacio de trabajo. En este modo, el robot nunca se mueve hasta que el operario se encuentre fuera del espacio compartido.
    2. **Guiado manual:** en este modo el robot es movido entre diferentes posiciones por el operario, con el fin de crear una nueva programación, ajustar trayectorias o enseñar nuevas trayectorias. Mientras se encuentre en este modo, el robot únicamente se moverá cuando el operario lo guie.
    3. **Monitorización de velocidad y separación:** tanto el robot como el operario pueden estar activos a la vez en un mismo espacio de trabajo, pero el robot ha de controlar en todo momento cuál es la distancia a la que se encuentra del operario, y deberá ajustar la velocidad de sus movimientos de forma proporcional a la distancia, llegando incluso a detenerse si la distancia fuera suficientemente pequeña.
    4. **Limitación de potencia y fuerza:** es el modo más colaborativo, ya que tanto el robot como el operario pueden estar activos, gracias a que el robot limita la fuerza que ejerce de acuerdo con los umbrales de dolor y lesiones definidos en la misma norma.
- **ISO 13855:2024 – Seguridad de la maquinaria – Posicionamiento de las medidas de seguridad con respecto a la proximidad del cuerpo humano:** En esta norma se encuentra la fórmula matemática mediante la que calcular la distancia mínima a la que se debe encontrar un dispositivo de seguridad de la zona en la que se encuentra el peligro. La fórmula proporcionada es:

$$S = (K \times T) + D_{DS} + Z$$

Donde:

- **S** es la distancia de separación en milímetros, nunca inferior a 100 mm [24].

- **K** es un parámetro derivado de las velocidades de aproximación de las partes del cuerpo humano, y que toma valores estandarizados de 2000 mm/s para los miembros superiores y de 1600 mm/s para el avance a pie [24].
- **T** es el tiempo de reacción del sistema en segundos. Este puede ser tanto medido como calculado. Para el cálculo de este se utiliza la fórmula $T = TSRP / CS + TME + TF,$ en la que TF es el factor de tolerancia, TME el tiempo hasta parada de la máquina y TSRP / CS el tiempo de reacción de los dispositivos de seguridad [24].
- $\mathbf{D_{DS}}$ es el valor máximo que pueden tomar unos suplementos para el alcance de un sistema de protección [24].
- **Z** es un parámetro que determina el factor de separación adicional, que depende de la aplicación y de factores como la incertidumbre en la medición [24].

- **ISO 13857:2019 – Seguridad de la maquinaria – Distancias de seguridad para evitar el alcance de zonas de peligro con extremidades superiores e inferiores:** Esta norma es un complemento a la anterior, y establece los valores de distancias de seguridad a cumplir para impedir que una persona entre con sus extremidades, tanto superiores como inferiores, una zona determinada como peligrosa, ya sea por encima de la estructura de protección, por una apertura o cualquier otro acceso. Las distancias exigidas son variables en función de las características de los elementos de protección.
- **Marcado CE:** Además de las normativas técnicas previas, la instalación también debe ser realizada conforme al marcado CE para instalaciones en la unión europea. En este caso aplica la Directiva de Máquinas 2006/42/CE [23], que será sustituida por el Reglamento de Maquinaria (UE) 2023/1230 a partir de enero de 2027 [23]. Este se trata de un requisito legal necesario para que la célula pueda ponerse en funcionamiento o ser comercializada. Este marcado certifica que la maquinaria cumple con todos los requisitos de salud y seguridad de esta normativa. El proceso general incluye la evaluación de riesgos la normativa ISO previamente descrita, la elaboración de una documentación y el firmado de la Declaración de Conformidad UE [23].

3.4.2. Cumplimiento de la normativa

Una vez está descrito el marco normativo a cumplir, en este apartado se detallará cómo se han tenido en cuenta cada una de estas normativas a la hora de diseñar los diferentes elementos de la célula.

En primer lugar, en cumplimiento con la norma ISO 12100, se ha realizado un estudio de los posibles peligros que pueden surgir durante la operación de la célula, siendo estos el peligro de atrapamiento o golpeo a un operario por parte del brazo robótico en sus movimientos y el atrapamiento en los mecanismos de una de las cintas transportadoras de un operario.

Siguiendo con el cumplimiento de las normas ISO 10218-1 y ISO 10218-2, se ha realizado un estudio de seguridad sobre el propio layout del proceso, sobre el cual se han acotado las diferentes zonas de peligro y estas han sido clasificadas en una escala de colores según la peligrosidad que estas suponen (Figura 15). La zona de mayor peligro es aquella en la que opera el robot UR5e, ya que en ella es donde el efector final del robot se situará y alrededor de la cual se llevarán a cabo todos sus movimientos. Esta zona no coincide al completo con la zona de alcance del robot, ya que mediante la definición de límites de los ejes del robot, el movimiento de este será sólo permitido dentro de esa zona restringida.

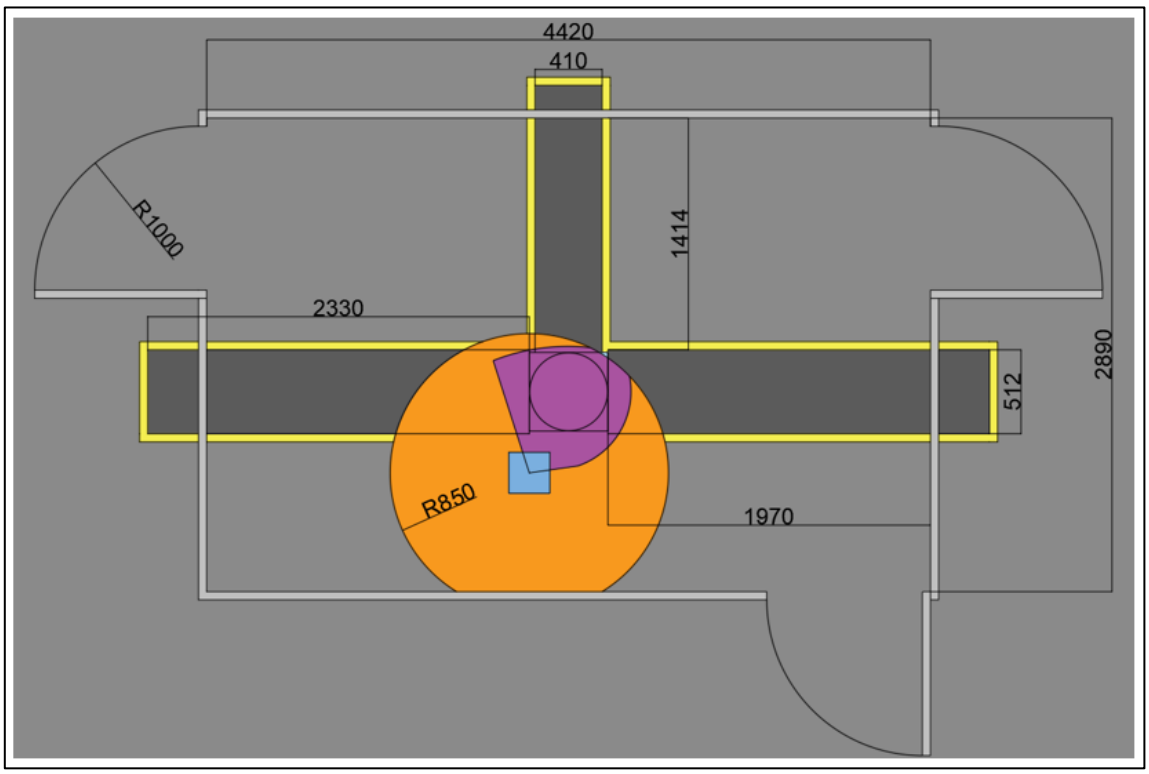


Figura 14: Layout acotado con las distintas zonas de peligro distinguidas por colores

En la figura se pueden observar 3 zonas de peligro, en amarillo se encuentra la zona inmediatamente próxima a las cintas transportadoras, en esta zona únicamente se pueden dar casos de atrapamientos por enganches en la ropa de los operadores, ya que la distancia entre la carcasa de la cinta y su elemento móvil es menor de 25 mm, por lo que según la normativa ISO 13854 [25], no hay posibilidad de que se den atrapamientos de ninguna parte del cuerpo. En naranja se ve representada la zona de alcance total del robot, la cual, aun no siendo utilizada durante la inspección, ha de tenerse en cuenta igualmente, ya que aunque el espacio de trabajo de robot se vea limitado por software, esta no es una medida de seguridad suficiente para considerar el espacio completamente seguro. Por último, en morado se encuentra la zona de trabajo del robot previamente explicada, dentro de la que este llevará a cabo todas sus tareas de inspección, por lo que supone la zona de mayor riesgo dentro de esta célula.

En cuanto a la protección y la delimitación del perímetro, se ha optado por un vallado de seguridad industrial BRÜHL FLEX II, el cual cumple con la normativa ISO 13857. Además, se han incluido 3 puertas de la misma serie para facilitar el mantenimiento de la célula. Para calcular la distancia mínima a la que se deben situar estas puertas, se ha realizado un estudio siguiendo la norma ISO 13855. Según el manual de usuario del UR5e [20], el límite de tiempo de parada es de 50 ms, a los cuales hay que sumar 524 ms que tarda la parada en quitar completamente la alimentación del robot. Teniendo en cuenta que se trata de una puerta que se debe atravesar andando, pero que además al tratarse de una valla de poca altura, se puede alcanzar la estación con las extremidades superiores, la velocidad de acercamiento para tener en cuenta para los cálculos será de 2000mm/s, y considerando el tiempo de detección de la puerta despreciable, realizando el cálculo de la fórmula:

$$S = (2000 * 0{,}574) + 0 = 1148\,mm$$

Se obtiene como resultado una distancia mínima de 1148 mm, que como se puede ver representado por el círculo de color verde en la siguiente imagen (Figura 17) es cumplido por todas las puertas de seguridad de la celda.

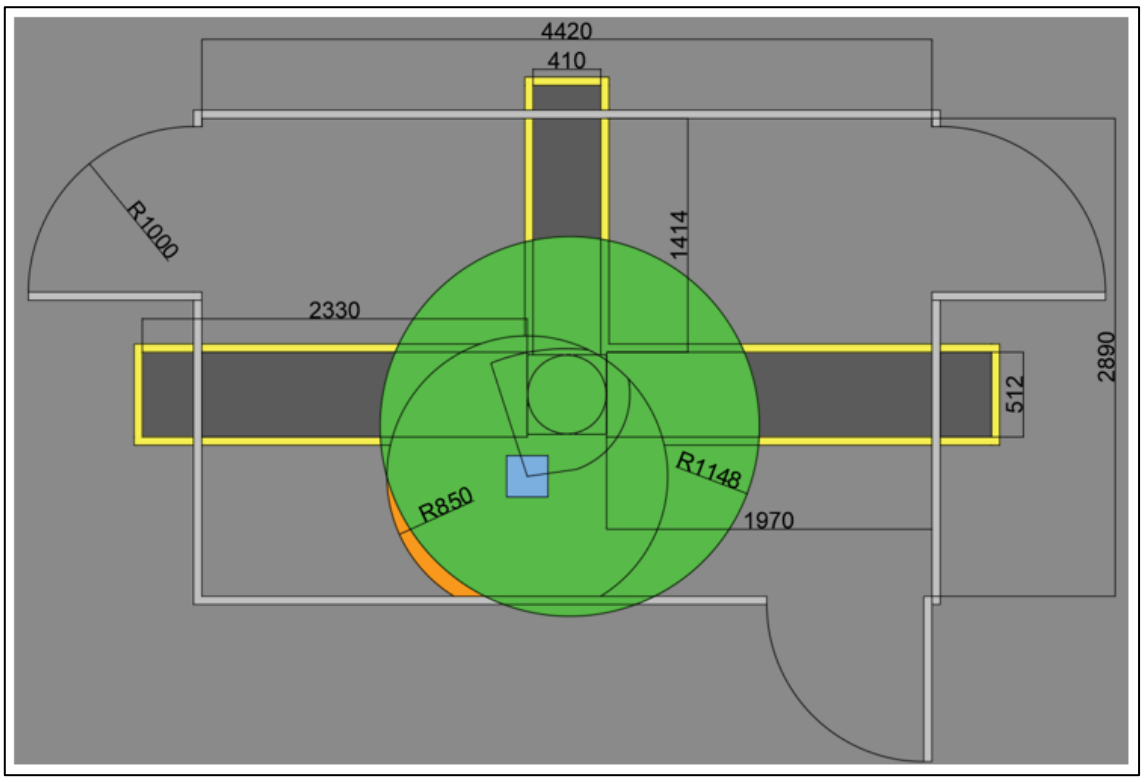


Figura 15: Fórmula de la distancia de elementos de seguridad aplicada a la célula

Por último, la aplicación de todas estas medidas y, por tanto, el cumplimiento de la normativa ISO, constituye el primer paso hacia la obtención del marcado CE para esta instalación, quedando pendiente la finalización de la documentación al completo y la formalización de la Declaración de Conformidad UE.

3.5. Estudio de ciberseguridad

3.5.1. Normativa aplicable

Además de la seguridad física en planta, la estación también debe tener una serie de medidas de ciberseguridad que lo protejan en caso de ataques por parte de terceros. Estas medidas serán aplicadas de forma acorde a una serie de normas:

- **IEC 62443:** Es una serie de normas desarrollada para la ciberseguridad en los sistemas de automatización y control industrial (IACS) [26]. Esta norma está estructurada en 4 categorías principales, estas son:
  - **IEC 62443-1**: la categoría general, en esta se sientan las bases sobre las que se desarrolla el resto de la norma. Establece conceptos como el modelo de zonas y conductos, que consiste en la división de la estación en agrupaciones con requisitos de seguridad similares, y el de los 4 niveles de seguridad (SL) [26]:

Nivel	Descripción
SL1	Protege contra errores humanos o accidentales, no maliciosos
SL2	Protege contra ataques maliciosos realizados usando métodos sencillos, por individuos con bajos recursos, capacidades o motivaciones
SL3	Protege contra ataques realizados por usuarios con recursos moderados y motivados que utilizan herramientas avanzadas
SL4	Protege contra ataques con medios avanzados con capacidad destructiva

Tabla 1. Clasificación de los niveles de Ciberseguridad según IEC 62443-1. Creada a partir de [26]

  - **IEC 62443-2:** Esta está más orientada hacia el propietario, es decir, la empresa que instalará la estación en su planta. En esta se guía el establecimiento de un programa de ciberseguridad válido mediante el uso de políticas procedimientos y prácticas de gestión [26].
  - **IEC 62443-3:** Centrada en el diseño técnico del sistema, guía el análisis de riesgos y la definición de diferentes zonas a las que establecer un nivel de seguridad común dependiendo de las consecuencias de un posible ataque. Además, también explora los requisitos de seguridad en un sistema, siendo estos: control de identificación y autenticación, control de uso, integridad del sistema, confidencialidad de los datos, restricción del flujo de datos, respuesta oportuna a eventos y disponibilidad de recursos [27].
  - **IEC 62443-4:** Establece los requisitos que cada componente individual del sistema tiene que cumplir para ser considerados seguros. Establece el proceso que se debe de seguir en su fabricación y cómo incorporar los requisitos de seguridad de IEC 62443-3 en ellos [26].
- **IEC/TS 63074:** trata los aspectos de seguridad centrados en la seguridad de sistemas de control relacionados con la seguridad, es decir, cómo podría un ciberataque influir en la seguridad establecida en las propias funciones del robot, como paradas de emergencia o límites de velocidad [28].

- **Reglamento (UE) 2019/881:** marco general europeo para certificar la ciberseguridad de productos, servicios y procesos tecnológicos [29].
- **Reglamento de Maquinaria (UE) 2023/1230:** sustituirá a la Directiva de Máquinas 2006/42/CE [23] a partir de enero de 2027, y incluye nuevos requisitos de ciberseguridad para la obtención del marcado CE.

3.5.2. Cumplimiento de la normativa

A la hora de la realización de este proyecto, se ha considerado la división por zonas establecida en la norma IEC 62443, de esta forma se ha obtenido el siguiente diagrama de red (Figura 18):

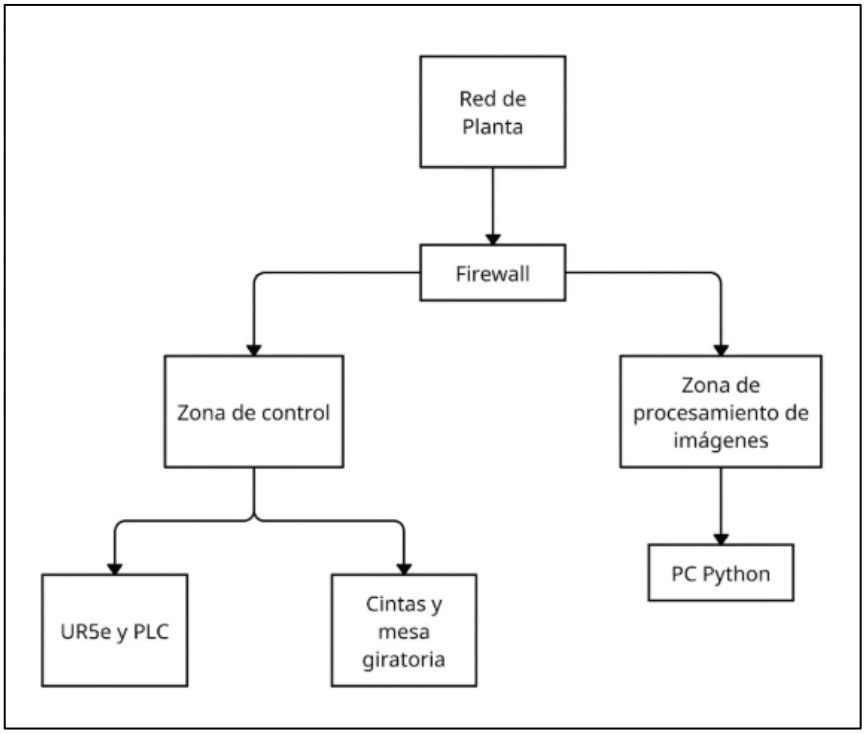


Figura 16. Diagrama de red de la estación

Además, se cumplen todos los requisitos de seguridad establecidos en la norma IEC 62443-3. Esto se logra gracias a:

- El uso de accesos mediante usuario y contraseña tanto al ordenador, como al robot y los sistemas de control.
- La limitación de las funciones disponibles para cada usuario.
- La protección del software de control y de los programas de Python contra modificaciones no deseadas o autorizadas.
- La protección de las comunicaciones y los datos de inspecciones.
- La separación de las diferentes zonas de la estación.
- El registro de errores y la generación de alarmas ante fallos.
- La creación de copias de seguridad y la posibilidad de recuperación de los programas de control.

En relación con la regla IEC/TS 63074, se debe impedir que una modificación del sistema de control pueda modificar parámetros de seguridad de alta importancia de la célula, como las funciones de parada de emergencia del robot y la velocidad de sus movimientos, o la detección del acceso al perímetro vallado. Por ello, se tomarán medidas como la restricción del acceso a los sistemas de programación, la protección los parámetros de seguridad, la gestión los usuarios y la segmentación de comunicaciones.

Y por último, el Reglamento (UE) 2019/881 ha sido considerado como referencia de ciberseguridad para todos los elementos tecnológicos utilizados en la planta, seleccionando componentes y sistemas con mecanismos y certificaciones de ciberseguridad adecuadas al nivel de riesgo de la tarea que desempeñan.

3.6. Integración en el simulador

Una vez diseñada la estación al completo, con todos sus elementos y manteniendo el cumplimiento de las normativas, se lleva a cabo la integración de esta estación dentro del simulador. Para ello, se han utilizado tanto elementos de la propia librería de RoboDK como modelos y mecanismos propios.

En primer lugar, el robot UR5e, el pedestal sobre el que se monta, la cámara con la que toma las imágenes de los objetos a escanear, y las cintas, tanto de entrada como de salida, fueron importados desde la librería de elementos del propio RoboDK [30]. Esto permite una rapidísima importación desde la propia opción *Open* con la que cuenta la página web, la cual carga el objeto seleccionado dentro de una nueva estación de RoboDK, o de la estación que se encuentre activa en el momento de seleccionarlo. Además, la librería también permite la descarga de los objetos, en caso de que se requiera su uso en diferentes ocasiones, permitiendo importarlo así directamente desde el almacenamiento del propio ordenador.

En el caso del robot y la cámara, no se realizó ninguna modificación sobre el modelo original, pero para conseguir que las cintas transportadoras y el pedestal contaran con las medidas correctas dentro del layout, a estos elementos se les realizaron una serie de operaciones. En el pedestal, basta con realizar un escalado en Z, pero para aplicar este escalado únicamente en un eje, se ha de utilizar la ejecución de guiones incluida en RoboDK (Figura 17). Haciendo uso de esta función se pueden ejecutar una serie de instrucciones tanto sobre la estación como sobre cada uno de los elementos dentro de esta. Para este caso, el guion más conveniente es *"ScaleXYZ"* (Figura 17), que permite el escalado independiente en cada uno de los ejes de un objeto seleccionado. Dentro de este guion, se elige el objeto o herramienta al que se le quiera aplicar y por último, se escriben los valores de escala deseados en un array, en orden [x, y, z].

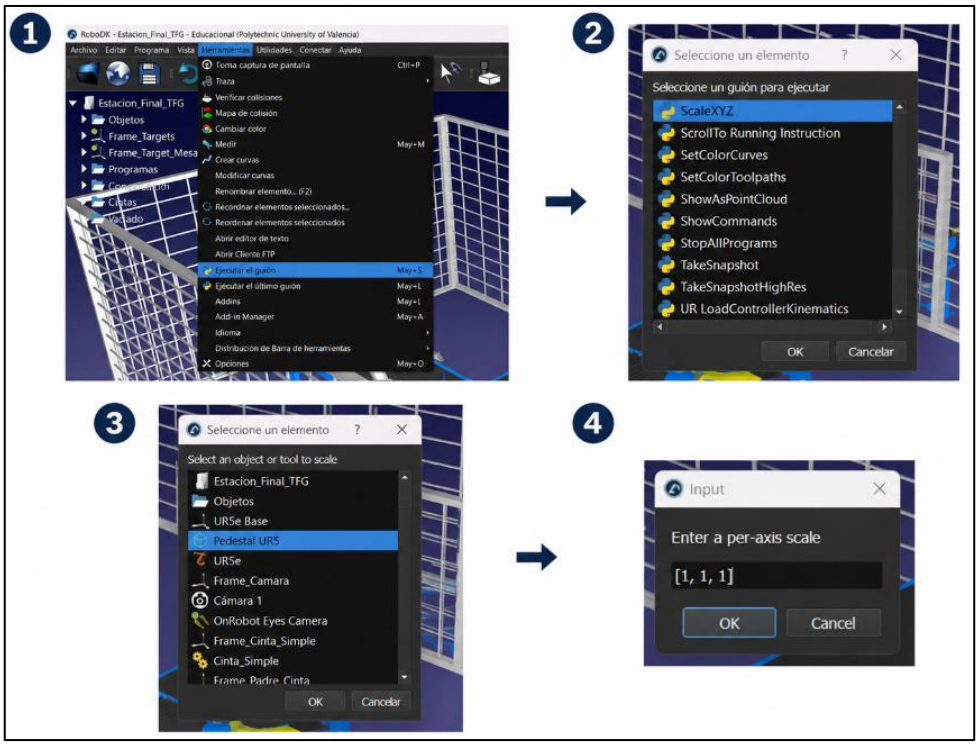


Figura 17. Pasos a seguir para el escalado de un objeto

No obstante, no se pueden realizar escalados sobre mecanismos como las cintas de forma directa, ya que lo único a lo que se le aplica el escalado en ese caso es a la geometría visual del mecanismo, no a sus elementos de simulación, por lo que no serían utilizables. Para resolver este problema, primero se entra en la opción de modificar el robot (o mecanismo) (Figura 18), lo cual generará un nuevo frame en el que encontraremos por separado el mecanismo y el objeto visual del mismo, el cual podremos convertir en objeto 3D, siendo este objeto el usado para la operación de escalado, y que deberá ser situado en el mismo frame que el mecanismo a modificar. Después se cierra la pestaña de modificación y se escala el objeto convertido a 3D. Una vez escalado, seleccionaremos el elemento original y volveremos a modificarlo, dentro de la pestaña de modificación, se ha de seleccionar el frame en el que se encuentran ambos elementos para que el mecanismo se mantenga en su lugar y no se den fallos de calibración. Por último se establece el objeto escalado como base y la *joint* generada al abrir la ventana de modificación como *joint 1*, y se confirman los cambios, obteniendo así el nuevo mecanismo con la geometría deseada para el proyecto.

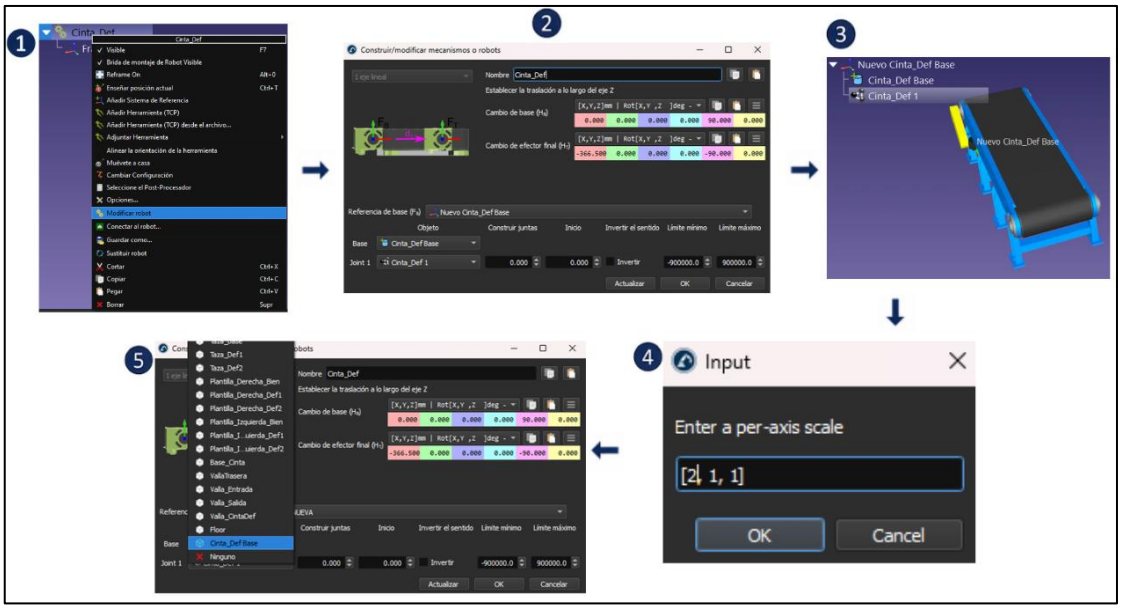


Figura 18. Pasos a seguir para la modificación de la geometría de un mecanismo

Para los elementos específicos personalizados para esta aplicación, se realizó un modelado en 3D utilizando la aplicación Autodesk Inventor. Estos elementos incluyen el vallado de la estación, la mesa rotatoria y la cinta que gira sobre esta, y los objetos a escanear. Una vez modelados, estos objetos, originalmente en formato .ipt de Inventor, fueron exportados como archivos de objeto 3D .stl, un formato compatible con la importación de objetos dentro de RoboDK.

En el caso de los objetos a escanear y las vallas, estos simplemente se importaron y se situaron en un frame auxiliar, desde el cual se crearán las copias aleatorias que serán visibles en las cintas durante el proceso. A estos objetos se les aplicó un cambio de color utilizando la herramienta de cambiar color de RoboDK y seleccionando las superficies que lo necesitaran.

En cuanto a la mesa rotatoria y la cinta que gira sobre esta, se tuvo que llevar a cabo el modelado de dos mecanismos independientes utilizando la utilidad de modelizado de mecanismos o robots que podemos encontrar dentro de la pestaña de utilidades integrada en el mismo simulador. El mecanismo de la mesa rotatoria cuenta con un único eje rotativo, mientras que el mecanismo de la cinta que rota sobre la mesa cuenta con un único eje lineal. Para llevar a cabo el modelado de estos mecanismos se siguió el siguiente procedimiento:

- Primero, se modeló el mecanismo de la mesa rotatoria, para esto, primero se debe abrir la ventana de modelizado de mecanismos o robots, dentro de la cual se puede elegir el tipo de articulación con la que contará el mecanismo y la cantidad con las que cuenta. Dentro de esta ventana, se elige también cuál será el objeto que servirá como base visual del mecanismo, y cuál será el que sirva como efector final. En este caso, el accionador es distinto a la base, por lo que se seleccionan ambos elementos por separado, previamente situados de forma correcta en la estación, cada una en su correspondiente frame. A continuación se

deben definir los parámetros de funcionamiento de cada una de las articulaciones del mecanismo, como en este caso el límite de grados de giro que puede alcanzar en ambas direcciones. Por último, hay que aplicar un cambio de base, ya que si no, el efector final del mecanismo se encontraría en el lugar de la base de este, así que en este caso se da un desfase de 280mm en el eje z, situando el efector final de la mesa justo encima de la plataforma giratoria en su centro. Así, obtenemos el siguiente cuadro final (Figura 19), que tras clicar en OK dará como resultado el mecanismo generado.



Figura 19. Ventana de creación de mecanismos para la mesa rotatoria

- Después, ya completado el modelado de la mesa, se modeló el mecanismo de la cinta que se monta sobre esta. El procedimiento principal es igual al del modelado de la mesa con algunas modificaciones. En este caso, el modelado se llevará a cabo utilizando únicamente un objeto 3D, ya que la carcasa exterior se implementará de forma independiente más adelante para evitar que esta rote. En este caso se selecciona la opción de ejes correspondiente y únicamente se añade el objeto como base. Se establece el cambio de base necesario con la altura de los rodillos de la cinta y los límites de movimiento en ambas direcciones para evitar fallos durante la simulación. Así, se obtiene la siguiente ventana.



Figura 20. Ventana de creación de mecanismos para la cinta

- Una vez modelados ambos mecanismos, el mecanismo de la cinta es añadido al de la mesa en relación padre-hijo en forma de hijo, haciendo que siempre que esta cambie de posición, la cinta rote con ella. Así, se obtiene la siguiente jerarquía dentro de RoboDK (Figura 21) y se concluye el modelado del mecanismo como tal.



Figura 21. Jerarquía del mecanismo tras añadir la cinta

- Para hacer que la simulación sea visualmente fiel a la realidad, se ha de añadir la estructura exterior de la cinta rotatoria, la cual la protege ante polvo y otros elementos, además de proteger a los operarios de posibles atrapamientos. Para añadir esta, simplemente se sitúa su objeto 3D dentro del mismo frame que la mesa rotatoria al nivel del padre, obteniendo la siguiente jerarquía (Figura 22). Además, se le da una posición relativa con respecto de este frame. De esta forma, se evita que los movimientos del efector final del mecanismo afecten a este objeto, pero manteniendo una posición relativa a la base de la mesa constante, dando como resultado una representación visual fiel del comportamiento del mecanismo. Así, obtenemos como resultado el mecanismo completo (Figura 8) para ser situado en el layout.



Figura 22. Jerarquía del mecanismo completo

Ya con todos los elementos importados, todos los mecanismos y objetos fijos son situados en frames independientes, esto se hace para poder establecerlos en posiciones relativas al origen de la estación y así tener una base desde la que introducir nuevos objetos de forma más fácil. Además, estos frames individuales son especialmente útiles a la hora de generar objetivos de posiciones para el robot, ya que haciendo uso de estos, se pueden situar estos objetivos de forma que el robot apunte perpendicularmente a un objeto o se sitúe justo sobre el centro de la mesa rotatoria.

3.7. Calibración de la cámara y programación del procesamiento de imágenes

Antes de comenzar con la programación de los movimientos del robot o del comportamiento de la célula, se llevó a cabo la calibración de la cámara y la creación de los scripts de toma de imagen y procesamiento de estas. La cámara simulada cuenta con una longitud focal de 5 mm, un campo de visión de 50 grados y una distancia de trabajo de 500 mm. El tamaño del sensor de la cámara, y por tanto el de la ventana de la toma de imágenes, es de 420 x 420 píxeles.

A la hora de la toma de imágenes para procesar la cámara se encuentra situada sobre el centro de la cinta rotatoria, y, para las imágenes tomadas de forma vertical, a una altura de 420 mm sobre esta. Para realizar la toma de imágenes del cubo, la cámara siempre se mantiene en perpendicular a una distancia de 230 mm de la cara que se está escaneando en ese momento, menos en el caso de la cara superior, ya que para evitar que el robot detecte patrones no deseados por los rodillos de la cinta, este toma la imagen superior a una distancia de 220 mm, reduciendo de esta forma en gran medida la influencia de los rodillos en la detección de patrones.

En cuanto a la taza, la imagen es tomada en posición horizontal de forma perpendicular al diseño impreso en esta, y a una distancia de 100 mm. Para lograr esto, la cámara se sitúa a 80 mm del borde de la cinta rotatoria y esta acerca la taza para lograr así tomar la imagen con el máximo nivel de detalle posible.

3.7.1. Procesamiento de las plantillas

Una vez establecidas todas las posiciones de toma de imágenes, se pasó al diseño del algoritmo para el procesamiento de las mismas, y para ello, se comenzó desde la forma más sencilla, las plantillas. El procesamiento de estas se lleva a cabo en el programa *Comp_Plantilla.py*. Este programa comienza con la carga de dos imágenes de referencia, una para cada tipo de plantilla, izquierda o derecha.

Una vez cargadas las imágenes, se transforman para convertirla en blanco y negro puros, facilitando así la búsqueda de contornos. El siguiente paso que realiza el programa es la búsqueda del contorno principal de la plantilla, lo cual es realizado en la función *obtener_contorno_principal(img_bw) (Figura 23)*, en la que se hace uso de la función *findContours* de OpenCV. Usando el resultado de esta función se devuelve el contorno de mayor tamaño encontrado en la imagen.

```
def obtener_contorno_principal(img_bw):
    # RETR_EXTERNAL busca únicamente el contorno exterior más externo
    cnts, _ = cv.findContours(img_bw, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
    return max(cnts, key=cv.contourArea) if cnts else None

ref_izq = obtener_contorno_principal(img_izq)
ref_der = obtener_contorno_principal(img_der)
```

Figura 23. Función obtener_contorno_principal

A continuación, se realiza la captura de un snapshot haciendo uso de la función de RoboDK *Cam2D_Snapshot*, lo cual da como resultado un array de bytes, que utilizando la función *cv.imdecode* de cv2, se convertirá en la imagen resultado decodificada y posteriormente convertida a escala de grises. Después se le aplica la función *obtener_contorno_principal* a la imagen obtenida, consiguiendo así el contorno de la plantilla a clasificar, el cual se comparará con ambas imágenes de referencia con el fin de encontrar cuál es la que menor diferencia tiene. Esto se consigue haciendo uso de la función *cv.matchshapes* de cv2, la cual compara los *momentos de hu* de ambas.

Por último, se compara la diferencia obtenida entre las dos imágenes con el umbral de aceptación, el cual es establecido para cada uno de los objetos en un valor específico. Según la decisión que tome el programa, la imagen se verá representada en una ventana informativa sobre el programa. En caso de que la pieza sea correcta, se mostrará un mensaje sobre la imagen en color verde anunciándolo, y el contorno principal de la plantilla será dibujado en verde sobre esta, pero en caso de que sea incorrecta, tanto el texto como el contorno serán dibujados en color rojo. En ambos casos, se mostrará junto a la ventana de la imagen capturada una ventana con la imagen de referencia y su contorno principal encontrado (Figura 24). Esto último es especialmente útil en las fases de calibración para detectar posibles errores en la búsqueda de patrones y poder corregir los umbrales de aceptación y de separación de contornos.

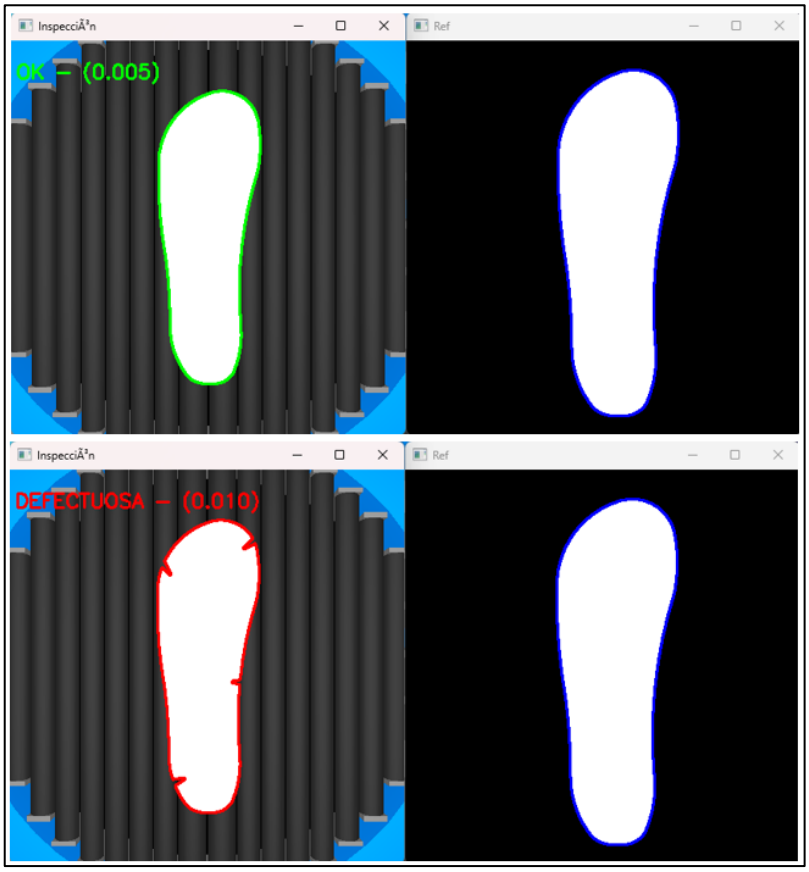


Figura 24. Ventana de comparación de imágenes de las plantillas. Arriba plantilla correcta y abajo defectuosa

3.7.2. Procesamiento de los cubos

Para la inspección de los cubos, el procesamiento de las imágenes es similar, pero el procedimiento de captura es más complejo. Para esto, se cuenta con dos programas para realizar la captura, estos son *Comp_Cubo.py*, encargado de la inspección individual de las caras del cubo, una vez situada la cámara en el lugar deseado, y *Cubo_General.py*, el cual está encargado de la sincronización de todos los elementos involucrados en la toma de estas imágenes.

En *Comp_Cubo.py*, el procedimiento a llevar a cabo es muy similar al de *Comp_Plantilla.py*, ya que el sistema de detección de patrones es también mediante el uso de *cv.findContours*. Sin embargo, para este caso, la función *obtener_contorno_principal* ha sido modificada a obtener*_contornos_internos* (Figura 25) de forma que los contornos encontrados son almacenados de forma independiente para su posterior procesamiento individual. Esto es así ya que en cada cara se encuentran formas diferentes separadas unas de otras, por lo que usar un único contorno no es posible a la hora de identificar defectos. De esta forma, se obtiene una lista con las figuras que serán comparadas posteriormente.

```
def obtener_contornos_internos(img_bw):
    cnts, _ = cv.findContours(img_bw, cv.RETR_LIST, cv.CHAIN_APPROX_SIMPLE)

    figuras_internas = []
    for c in cnts:
        area = cv.contourArea(c)
        if 4000 < area < 40000:
            figuras_internas.append(c)

    return figuras_internas
```

Figura 25. Función obtener_contornos_internos

Además, este programa también cuenta con la función *calcular_hu_global* (Figura 26), la cual extrae los *momentos de hu* de los contornos detectados en la imagen, permitiendo analizarlos de forma independiente. En esta función, primero se crea una imagen negra del mismo tamaño que la imagen a analizar, y sobre esta dibuja en blanco los contornos obtenidos. Después, mediante el uso de *cv.moments* se obtienen los momentos de la imagen de máscara, y utilizando la función *cv.HuMoments* se calculan los *momentos de hu* a partir de los anteriores, haciendo uso del método *flatten* para convertir el resultado en un array de 7 elementos. Por último, se aplica una transformación logarítmica al resultado, ya que los *momentos de hu* suelen tomar valores muy pequeños con grandes distancias entre ellos. De esta forma los datos serán más fáciles de tratar y de interpretar en los pasos posteriores.

```
def calcular_hu_global(img_bw, contornos):
    mascara = np.zeros_like(img_bw)
    cv.drawContours(mascara, contornos, -1, 255, -1)
    momentos = cv.moments(mascara)
    hu = cv.HuMoments(momentos).flatten()
    hu_log = -np.sign(hu) * np.log10(np.abs(hu))
    return hu_log
```

Figura 26. Función calcular_hu_global

Con estas funciones, se obtienen los datos necesarios para realizar la comparación y la clasificación de las imágenes, la cual es realizada de forma similar a las de las plantillas. Una vez se tiene la imagen en escala de grises, se obtienen sus contornos y se consigue el valor de los *momentos de hu* usando las funciones anteriores. A continuación, se comparan los valores de los *momentos de hu* entre la imagen tomada y la imagen de referencia para la cara actual, y de igual manera que en el caso de las plantillas, se muestra el resultado en dos ventanas, una con la imagen tomada y sus contornos dibujados, o bien en verde si es correcta, o en rojo si es defectuosa, y en otra la imagen de referencia con sus contornos dibujados en azul independientemente del resultado de la inspección (Figura 27).

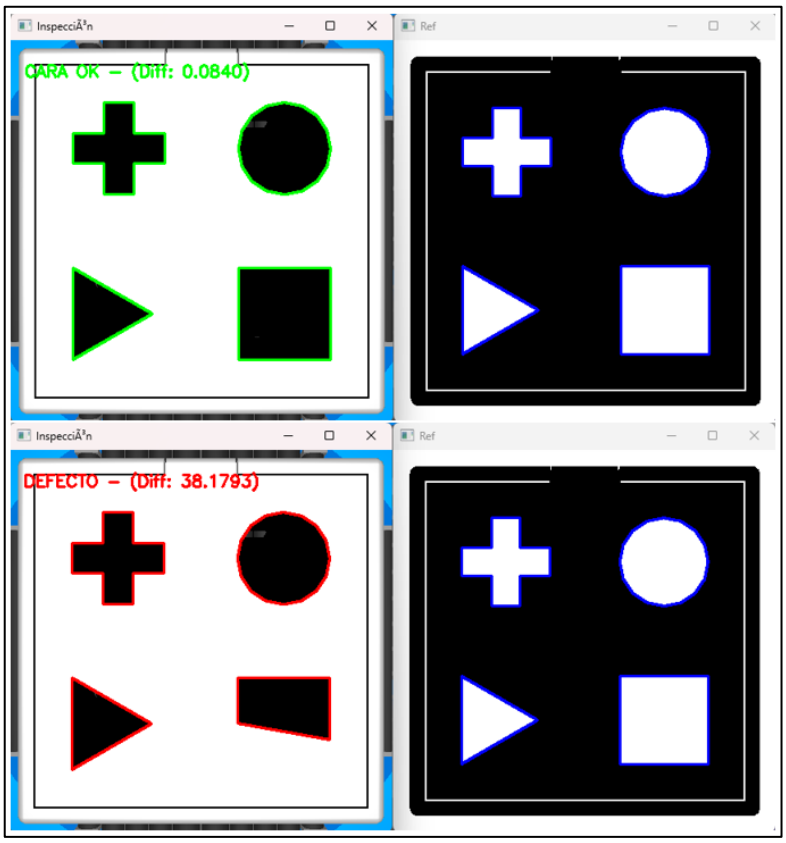


Figura 27. Ventana de comparación de imágenes de los cubos. Arriba cubo correcto y abajo defectuoso

El otro programa encargado de la inspección de los cubos es *Cubo_General.py*. En este, primero se cogen todos los elementos de la estación que intervienen en este proceso y se guardan en variables igualando estas a la función *RDK.Item*. Esto incluye tanto los mecanismos que intervienen, como los frames y los targets correspondientes a la tarea, y los programas que se llamen durante la ejecución, como en este caso *Comp_Cubo.py*. Además, también se leen los parámetros de la estación necesarios mediante la función *RDK.getParam*, los cuales actúan como variables globales entre distintos programas para lograr la sincronización.

Una vez se cuenta con todos los parámetros y elementos necesarios, el robot se mueve a la posición de toma de imágenes inicial, una vez ahí, establece el parámetro de la

estación *'cara_actual'* a *'Cubo_Arriba'*, lo cual indica al programa de inspección llamado a continuación (*Comp_Cubo.py*) que la cara que deberá inspeccionar es la superior. Después, recupera el parámetro de estación *'defectuoso'*, y tras comprobar que el resultado es 0, repite la operación moviendo el robot y la mesa a las poses deseadas y llamando al programa de inspección cambiando el parámetro *'cara_actual'* al valor correspondiente en cada una de ellas. En el caso de que se encuentre un fallo en una de las caras del cubo, el programa deja de llevar a cabo la comprobación y el robot vuelve a la posición inicial, dando por concluida la inspección.

3.7.3. Procesamiento de las tazas

Al igual que en el caso de los cubos, para llevar a cabo la inspección de las tazas también se divide el proceso en dos programas. Estos son *Comp_Taza.py*, encargado de la toma y el procesamiento de las imágenes, y *Comp_Taza.py*, encargado de la sincronización entre los elementos para realizar la toma de estas.

El programa *Comp_Taza.py* es muy similar a *Comp_Cubo.py*, de la inspección de los juguetes para niños. Este programa comparte la función *calcular_hu_global* (Figura 26) con este programa, además, también cuenta con una versión modificada de la función *obtener_contornos_internos* (Figura 25) llamada *obtener_contornos_logo* (Figura 28). En esta nueva función, primero se detectan los contornos de la imagen de igual forma que en las versiones anteriores, mediante el método *cv.findContours*, pero en este caso la jerarquía obtenida es utilizada en la función. Esta jerarquía es necesaria para detectar únicamente los patrones internos de la taza, ya que de esta forma se puede evaluar de forma exclusiva si el logo se encuentra impreso de forma correcta.

```
def obtener_contornos_logo(img_bw):
    # RETR_TREE devuelve la jerarquía completa [Next, Previous, Child, Parent]
    cnts, hierarchy = cv.findContours(img_bw, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)

    figuras_logo = []
    if hierarchy is None:
        return figuras_logo

    hierarchy = hierarchy[0]

    for i, c in enumerate(cnts):
        area = cv.contourArea(c)
        if 60 < area < 30000:
            padre_idx = hierarchy[i][3]
            if padre_idx != -1:
                x, y, w, h = cv.boundingRect(c)
                proporcion = w / float(h)
                if proporcion < 3.0:
                    figuras_logo.append(c)

    return figuras_logo
```

Figura 28. Función obtener_contornos_logo

Mediante el uso de estas funciones, se lleva a cabo la comparación entre la imagen tomada y la imagen de referencia de la taza. Este se lleva a cabo de forma similar a las anteriores, primero, se toma la imagen usando el método *RDK.Cam2D_Snapshot*, la cual será convertida a escala de grises y binarizada para su procesado. Una vez se tiene la imagen preparada para el procesado, se obtienen sus contornos internos mediante el

uso de la función *obtener_contornos_logo* para luego calcular los *momentos de hu* con *calcular_hu_global*. Usando estos se lleva a cabo la comparación con los de la imagen de referencia, y si la diferencia entre ellas entra dentro del umbral establecido, se tomará como correcta. Para la visualización, esta se realiza de forma igual a las anteriores, mostrando en una ventana la imagen tomada con sus contornos resaltados en rojo o verde, dependiendo del resultado de la inspección, y con la imagen de referencia en otra ventana igualmente con sus contornos resaltados en color azul (Figura 29).



Figura 29. Ventana de comparación de imágenes de las tazas. Arriba correcta y abajo defectuosa

En el programa *Taza_General.py*, al igual que en *Cubo_General.py*, primero se recuperan todos los elementos de la estación necesarios para el proceso, y una vez estos se encuentran disponibles, se lleva a cabo la inspección de la taza. Primero se sitúa el robot en la posición de captura de imágenes de las tazas, pasando por dos posiciones intermedias para suavizar el movimiento y evitar posibles contactos no deseados con otras partes de la estación. Una vez el robot se encuentra en la posición, la mesa rotatoria gira la taza hasta que esta se encuentra con el logo visible para la cámara, y a continuación, usando la cinta, acerca la taza a la cámara. Después, se llama al programa *Comp_Taza.py* para que se lleve a cabo la inspección, y tras esperar a que finalice, tanto la mesa como la cinta y el robot vuelven a sus posiciones iniciales, preparándose así para la clasificación de la taza actual y para la llegada del próximo producto que llegue para ser escaneado.

3.8. Programación del robot, la mesa y las cintas

3.8.1. Programación del robot

Para encontrar las posiciones objetivo necesarias para la toma de imágenes, primero se eligió la posición en la que debería estar la cámara, y mediante una sencilla transformación, se halló la posición deseada para el robot. Tomando la geometría de la herramienta con la cámara (Figura 30), el desfase necesario con respecto al efector final del robot para que la cámara se encontrara en el lugar correcto para la toma de imágenes era de 34.5 milímetros en el eje Z y de -105 milímetros en el eje Y.

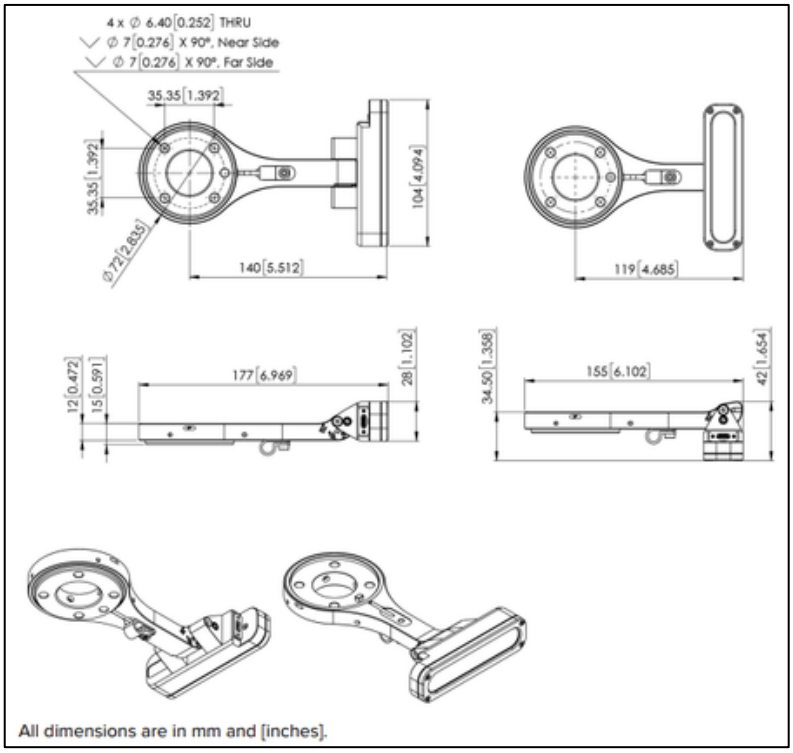


Figura 30. Planos con medidas en milímetros y pulgadas de la cámara Onrobot Eyes. Obtenida de [31]

Conociendo la transformación necesaria para hallar las posiciones del robot respecto a las de la cámara, se creó un frame principal dentro del cual se definirán todos los targets del robot para la toma de imágenes. Esto se realizó con el fin de aportar una mayor flexibilidad al programa, ya que si cambiara la ubicación de la mesa de inspección, simplemente habría que mover el frame de objetivos a la nueva posición de la mesa, manteniendo la distancia vertical hacia esta, ya que al pertenecer estos targets al frame, mantendrán siempre una posición relativa al mismo fija. Así, se obtiene la siguiente jerarquía de frame y targets para el robot (Figura 31).



Figura 31. Jerarquía de posiciones objetivo del robot

Una vez se encuentran definidas todas estas posiciones, el robot es controlado para que alcance estas posiciones a través de los programas de Python encargados de la sincronización de la estación.

Para asegurar el correcto funcionamiento del programa y evitar posibles colisiones o la generación de singularidades o posturas inalcanzables para el robot, es recomendable la creación de posiciones intermedias para que el movimiento del robot a la posición final se realice de una forma más controlada y se eviten trazadas con un mayor riesgo.

Además de la inclusión de posiciones intermedias, también es necesario comprobar que la configuración de las articulaciones del robot es correcta en cada una de las posiciones objetivo. Esto es necesario ya que en ocasiones la cinemática inversa del robot genera posturas en las que las articulaciones se encuentran muy cerca de sus límites rotacionales, lo que genera movimientos innecesarios a la hora de cambiar entre unas posiciones y otras, o incluso bloqueos en la ejecución del programa por intentos del robot de alcanzar posiciones imposibles.

Para conseguir esto, dentro de las opciones de cada uno de los elementos target de RoboDK, se encuentra el acceso a la ventana de Cambiar Config. (Figura 32). En esta ventana se permite elegir entre todas las configuraciones posibles de las articulaciones del robot que permiten que se alcance la posición deseada. Para facilitar la elección entre una de estas configuraciones, dentro de esta misma ventana se ofrecen 3 filtros que acotan la búsqueda a las configuraciones en las que se cumplen estas características. Estas son la rotación de la base del robot, a elegir entre frente o posterior, la posición del codo del robot, según se necesite que se encuentre arriba o abajo, y la configuración de la muñeca, ya sea en forma de muñeca invertida o no.



Figura 32. Ventana de configuración del robot para una posición objetivo

3.8.2. Programación de la mesa

A la hora de realizar la programación de la mesa, esta también ha sido realizada haciendo uso de los target de RoboDK. En este caso, los target afectan únicamente al elemento rotatorio de la mesa, ya que la cinta transportadora que se encuentra sobre ella es programada mediante el uso del simple programa *Avanzar_Cinta_Rot.py*, encargado de mover la cinta una cantidad de milímetros establecida tras comprobar que el mecanismo es válido.

En cuanto a los targets establecidos, estos son en primer lugar un target inicial *Init_Mesa*, que servirá como referencia de la posición desde la que arrancan todas las inspecciones y a la que deben volver una vez están completadas. A continuación, hay 3 posiciones encargadas de permitir el escaneo de todas las caras del cubo, estando estas situadas a 90 grados en torno al eje Z unas de otras. Para el escaneo de la taza, cuenta con una posición que sitúa la taza con el diseño apuntando directamente hacia la cámara. Por último, la mesa cuenta con dos posiciones dependiendo del resultado de la inspección del producto. Para los productos correctos, la posición de inicio es reutilizada, ya que simplemente basta con avanzar la cinta en esa posición, mientras que para los productos defectuosos, la mesa cuenta con target específico *Target_Def*, el cuál se encuentra situado a 90 grados en torno al eje Z de la posición inicial, alineando así la cinta con la de productos defectuosos.

3.8.3. Programación de las cintas

Además del movimiento de los objetos dentro de la estación, en esta simulación las cintas también están encargadas de la generación de nuevos objetos y son las que permiten el funcionamiento de toda la estación en sincronía.

Estas cintas son controladas por el programa *Generar_Objeto.py*, en el que mediante el uso de unos objetos auxiliares, situados en un frame oculto y con la visibilidad

desactivada, crea copias de forma aleatoria de los objetos cada vez que la cinta de entrada avanza y los sitúa de forma correcta en el centro de la cinta, haciendo que estos lleguen en la posición deseada a la mesa de inspección.

Para lograr esto el programa comienza estableciendo un valor de avance de la cinta para cada ejecución del programa, en este caso de 600 mm, y recuperando el valor del parámetro de estación *contador*, el cual lleva el registro de cuántos objetos han sido generados durante la ejecución del programa general. A continuación genera un número entero aleatorio entre 1 y 3, el cuál determinará qué tipo de objeto será generado en este paso, siendo 1 para una plantilla, 2 para un cubo y 3 para una taza. En caso de que el elemento a generar sea un cubo o una taza, se generará un nuevo número aleatorio, que determinará si tendrá defecto ese nuevo objeto y en caso de que lo tenga el tipo de defecto que tendrá. En el caso de que el primer número aleatorio generado sea un 2, se dará un paso previamente a esto, se generará un nuevo número entero aleatorio entre 1 y 2, decidiendo así si la plantilla es del lado izquierdo o del derecho.

Una vez se ha decidido el objeto a clonar, se toma el frame padre sobre el que se situará la copia como *frame_padre* y, antes de crear el nuevo objeto se crean tres variable con los posibles nombres que tendrá el próximo objeto que llegará a la cinta rotatoria, que se consiguen al añadir el valor del contador – 2 al nombre de cada tipo de objeto. Esto es necesario ya que dentro de este mismo programa, se llevará a cabo el cambio de jerarquía entre la cinta de entrada y la mesa de inspección, cambiando el padre del objeto que llega a la mesa para que el objeto se mueva junto a esta.

A continuación, si el mecanismo de la cinta es válido, se avanza el incremento de milímetros establecido haciendo uso del método *MoveJ*, después se copia el objeto padre, y se pega en una variable *copia*, a la que se establecerá como padre *frame_padre* para que esta se mueva con la cinta. Tras esto, se aumenta el valor del contador en uno, se sitúa la copia sobre la cinta usando el método *setPose* y se cambia el nombre de la copia al nombre del tipo de objeto más el valor del contador.

Una vez creada la nueva copia, se comprueba si alguno de los nombres creados para detectar cuándo llega un objeto a la mesa es correcto, y si se da el caso, se establece como padre de ese objeto la cinta rotatoria y este se sitúa en la posición y orientación correctas.

3.9. Sincronización y funcionamiento de la célula al completo

Para lograr la correcta sincronización y la facilidad de ejecución del programa, se ha creado el programa *Global.py*, el cual está encargado de la sincronización de todos los procesos, desde la entrada de nuevos objetos utilizando el subprograma *Generar_Objeto.py* hasta la eliminación de objetos ya escaneados y clasificados una vez estos salen de su cinta correspondiente, liberando así memoria y mejorando el rendimiento de la simulación.

A la hora de lograr la sincronización entre varios programas en una estación de RoboDK, resulta imprescindible hacer uso de los parámetros de estación que ofrece RoboDK. Estos son parámetros que son establecidos y recuperados, mediante el uso de las funciones de la API de Python *RDK.setParam* y *RDK.getParam*, durante la ejecución de los programas de la estación, y que pueden ser utilizados como variables globales, permitiendo de esta forma que los diferentes programas de la estación puedan mantener variables comunes, haciendo posible la comunicación entre ellos. Estos parámetros son definidos dentro de la ventana de Opciones dentro de la estación de RoboDK, en la pestaña de Estación (Figura 33).

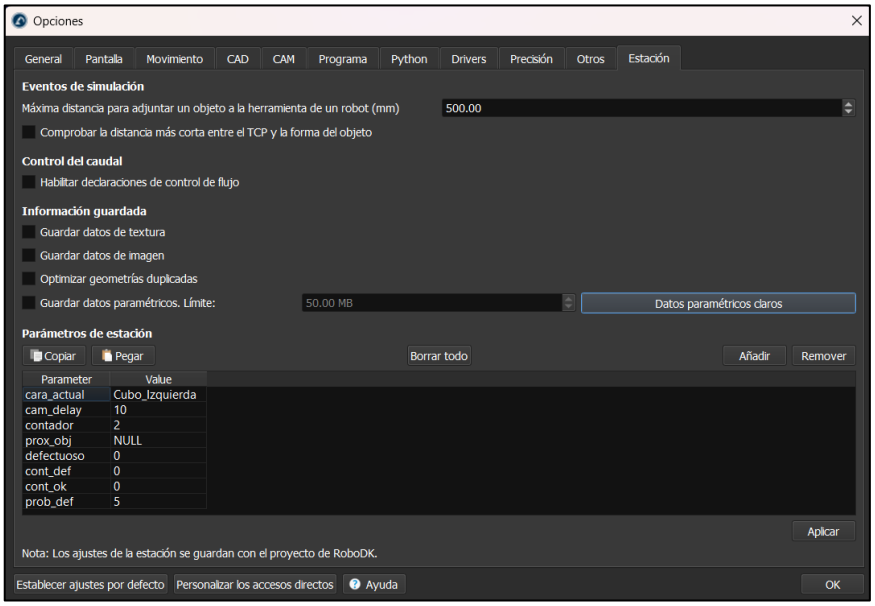


Figura 33. Pestaña "Estación" donde modificar los parámetros de estación

Este programa comienza haciendo uso del subprograma *Reset.py*. En este, se reestablecen los ejes de todas las cintas, además de eliminar todos los objetos generados a partir de los auxiliares y crear un nuevo objeto inicial en la cinta de entrada.

Tras esto, el programa comprueba que aún no ha llegado un objeto a la mesa de inspección, y mientras esto sea cierto, se crean nuevos objetos en la cinta de entrada. Una vez hay un objeto situado sobre la cinta de la mesa, esta se avanza usando el programa *Avanzar_Cinta_Rot*, y dependiendo del nombre del objeto que ha llegado se ejecuta el programa correspondiente para la inspección del objeto en cuestión.

Una vez completada la inspección del objeto, se recuperan los parámetros *defecto*, *contador*, *contador_def* y *contador_ok* de la estación. Si el valor de *defecto* es 1, esto significa que el objeto inspeccionado ha dado como resultado defectuoso, por lo que se mueve la mesa rotatoria a la posición de la cinta de defectuosos y avanza la cinta hasta que el objeto se encuentre sobre la cinta de salida de productos defectuosos.

Los parámetros *contador_def* y *contador_ok* son especialmente útiles a la hora de crear una simulación visualmente fiel, ya que haciendo uso de estos, es posible calcular la distancia a la que se debe situar el objeto del frame de la cinta para que la distancia entre

los objetos situados sobre esta se mantenga regular y no se den solapamientos. Además, esto permite también identificar el objeto que se debe eliminar una vez ha abandonado la cinta visualmente. Para esto, cuando un objeto es clasificado en cualquiera de las dos cintas, el nombre de este es cambiado a la combinación de 'Defectuoso' o 'OK', y el valor del parámetro *contador_def* o *contador_ok* correspondiente. De esta forma, cuando el valor de *contador_def* o *contador_ok* es mayor que 3, se elimina el objeto en la cinta correspondiente de cuyo nombre tenga el valor del respectivo contador – 3.

3.10. Planificación temporal

3.10.1. Planificación temporal del proyecto

Para el desarrollo de este proyecto, este ha sido organizado en una serie de fases secuenciales, aunque estas fases no han sido realizadas de manera completamente continua ni con una dedicación constante, sino en intervalos de mayor y menor dedicación, debido a la coincidencia del desarrollo de estas con periodos del curso académico con una mayor o menor carga de trabajo. Por ello, esta planificación temporal debe interpretarse como una secuencia lógica de fases, no como un ritmo de trabajo uniforme semana a semana.

La primera fase que se llevó a cabo fue un estudio del proyecto a realizar, fase en la que se analizaron los objetivos a alcanzar y los requisitos que la célula de inspección debía cumplir. Durante esta etapa se llevó a cabo también una primera aproximación al estado del arte y trabajos previos, revisando tecnologías y soluciones existentes en el ámbito de la inspección industrial por visión artificial y el uso de la robótica en este. Esta primera aproximación facilitó la elección tanto del equipamiento como del software a utilizar en el desarrollo del TFG.

A continuación, se abordó el diseño y el modelado en 3D en Autodesk Inventor de las piezas a escanear, en esta fase se definieron las geometrías, dimensiones y elementos característicos de cada una de las piezas, incluyendo los fallos de ejemplo de cada una de estas que deberían ser detectados por el sistema de visión.

Con las piezas ya definidas y modeladas, se llevó a cabo la configuración de la cámara virtual y la programación de los algoritmos de visión mediante Python con OpenCV. De forma paralela, se comenzó a realizar el diseño del layout, el cual es altamente dependiente de las piezas a escanear y de las técnicas de procesamiento utilizadas, ya que la inclusión de ciertos elementos y la distinta situación de estos puede influir en los posicionamientos del robot y en el reconocimiento de patrones.

Una vez definido al completo el layout final de la estación, se continuó con el modelado de los elementos restantes, como el vallado perimetral de seguridad de la celda, el cual se realizó en Autodesk Inventor, y la mesa rotatoria con su cinta, realizado en combinación de Inventor con la herramienta de modelado de mecanismos de RoboDK. Este paso resulta indispensable debido a que se trata de elementos completamente personalizados, por lo que no se encuentran en la biblioteca de RoboDK.

Con todos los elementos modelados de forma individual, se procedió a la integración de todos los elementos, tanto propios como importados, en una misma estación de RoboDK en la que se llevó a cabo la programación final de todos estos elementos y la sincronización entre ellos para permitir su trabajo en conjunto. Para esto se llevó a cabo la programación del robot y la mesa mediante targets de RoboDK, y la programación de las cintas transportadoras y de la sincronización general de la planta mediante el uso de la API de Python, dando así lugar al proceso de inspección al completo.

Una vez completa la implementación de la estación en RoboDK y la programación de esta, se llevó a cabo un periodo de pruebas y depuración de errores. Durante esta fase, se acabó de ajustar los parámetros de cada uno de los programas y se ejecutó de forma repetida y continua la estación, comprobando así el correcto funcionamiento de la estación y de todos sus elementos y de detectar posibles errores en la sincronización o la programación, con el fin de corregirlos hasta llegar a un estado final del sistema robusto y con una alta fiabilidad.

Por último, se llevó a cabo la redacción de la memoria y la documentación del proyecto. En esta fase se documentó el trabajo realizado a lo largo de todas las etapas previas, por lo que esta ha sido realizada de forma conjunta con el resto.

En la tabla siguiente (Tabla 2) se encuentran los tiempos dedicados de forma estimada para cada una de las fases anteriores:

Fase	Descripción	Duración Aproximada
1	Estudio del proyecto	1 semana
2	Diseño y Modelado 3D de las piezas	1 semana
3	Configuración de la cámara y programación de visión artificial	3 semanas
4	Diseño del layout	1 semana
5	Modelado de los elementos restantes de la estación	2 semanas
6	Integración de los elementos en el simulador y programación de la estación	3 semanas
7	Periodo de pruebas y corrección de errores	2 semanas
8	Redacción de la memoria	4 semanas

Tabla 2. Planificación temporal del proyecto con duración estimada en semanas

En total, la duración completa del desarrollo del proyecto ha sido de alrededor de 15 semanas, gracias a la posibilidad de llevar a cabo el desarrollo de fases como el diseño del layout y el modelado de elementos de la estación de forma conjunta, lo que ha acortado el tiempo final total empleado. Sin embargo, estas 13 semanas de trabajo no han sido realizadas de una forma continua, si no que estas han sido distribuidas de forma irregular a lo largo de 4 meses en función de la carga académica de cada uno de

estos, destacando el mes de julio y el inicio de agosto, en los cuales se encontraron los exámenes pertenecientes al segundo semestre del curso académico alemán.

Con el fin de representar de forma visual la distribución de estas fases a lo largo del desarrollo del proyecto, se ha creado un diagrama de Gantt en el que se representa la duración relativa y el posible solapamiento entre las distintas fases (Figura 34).

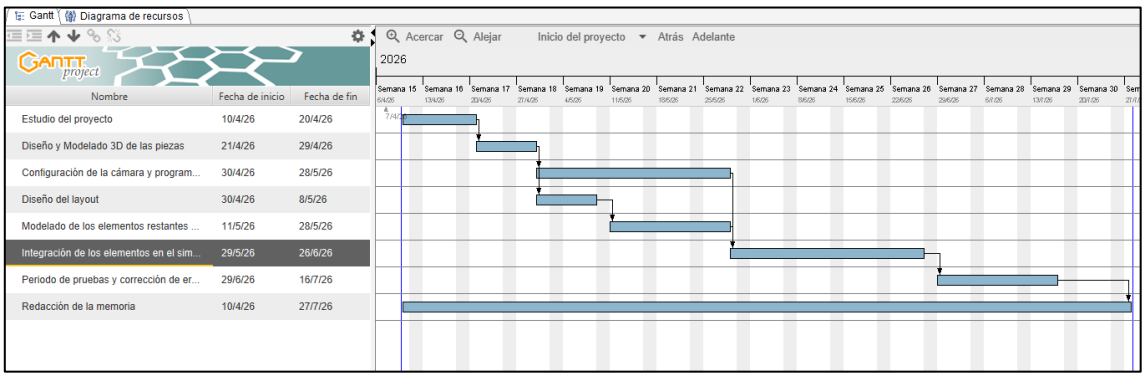


Figura 34. Diagrama de Gantt del Trabajo de Fin de Grado

3.10.2. Planificación temporal de instalación en planta

A la hora de realizar la instalación de esta solución en un entorno industrial, se plantea una segunda etapa de planificación temporal. En esta fase se encuentran todas las actividades necesarias para trasladar el diseño virtual desarrollado durante el proyecto a una instalación física, contando con todos los requisitos técnicos y de seguridad necesarios para llevar a cabo la instalación de forma correcta. Esta segunda etapa también se encuentra dividida en varias fases.

En primer lugar, se ha de realizar el estudio de ingeniería previa a la instalación del proyecto. Esta fase corresponde al desarrollo de todo el proyecto previo a la instalación, incluyendo todas las fases de elección de equipos, elección de softwares y el desarrollo del layout, cumpliendo con las normativas de seguridad necesarias. Además, también se tomarán en cuenta aspectos importantes como los espacios necesarios para mantenimiento, la accesibilidad a los equipos, la distribución de los elementos dentro de la zona de trabajo y la compatibilidad entre los distintos componentes de la estación. Durante esta fase, también se llevará a cabo la adquisición de los componentes necesarios, incluyendo todos aquellos elementos que resulten necesarios para la construcción de la célula.

A continuación, se realizará una preparación de la zona en la que se realizará la instalación posteriormente, adecuando así el espacio físico de la planta a las necesidades de la célula. En esta etapa se acondicionará el área destinada a la instalación, verificando que se dispone del espacio suficiente para llevar a cabo el montaje del layout respetando las distancias necesarias para el funcionamiento de forma segura de la estación y que permitan su mantenimiento. También se prepararían los distintos puntos de fijación de los equipos en el suelo para permitir el montaje de la célula.

Una vez preparado el espacio físico para el montaje de la estación, se procederá al montaje físico esta, comenzando por la instalación de la estructura principal como el vallado perimetral o las cintas y finalmente instalando el robot, la cámara y los distintos sensores. En esta etapa también se realizarán los ajustes mecánicos necesarios para asegurar la estabilidad de los elementos. Con el montaje completado, se procede a la configuración y la puesta en marcha de los equipos, verificando el correcto funcionamiento de cada uno de estos de forma independiente, con el objetivo de detectar problemas individuales de cada uno de los elementos y solucionarlos antes de llevar a cabo la coordinación de la estación al completo. Una vez verificado el correcto funcionamiento de todos los elementos, se continúa con la programación de la célula, incluyendo tanto la calibración y la verificación de los algoritmos de visión, como la programación del robot.

Tras esto, se procede con una fase de pruebas en la que se comprobará que todas las fases anteriores han sido realizadas de forma correcta y que no se han generado fallos, seguida de una puesta en marcha general en la que se comprobará le funcionamiento correcto de la estación. Por último, se incluye una fase de formación del personal, en la que se capacitará al personal encargado de la operación y el mantenimiento de la célula, con el objetivo de que los trabajadores conozcan su funcionamiento y puedan operarla de forma segura y adecuada. En la siguiente tabla (Tabla 3), se encuentran todas estas fases y sus respectivas duraciones estimadas en días.

Fase	Descripción	Duración Estimada
1	Estudio de ingeniería (el análisis y el desarrollo de la célula)	75 días
2	Preparación de la zona física para la instalación de la estación	5 días
3	Montaje físico de la estación	10 días
4	Configuración y prueba de los equipos individualmente	5 días
5	Calibración física del software de visión y programación del robot físico	20 días
6	Fase de pruebas y puesta en marcha	5 días
7	Formación del personal	3 días

Tabla 3. Planificación temporal de la instalación física del proyecto con la duración estimada en días

Las duraciones estimadas para cada una de las fases de instalación han sido estimadas mediante un criterio basado en la complejidad de cada una de las tareas y tomando como referencia los tiempos proporcionados por empresas de automatización en proyectos de tamaño y complejidad similares [32].

Para representar de forma visual la duración de la instalación de esta estación, se ha creado un diagrama de Gantt en el que se pueden ver representadas todas estas tareas y sus duraciones (Figura 35).

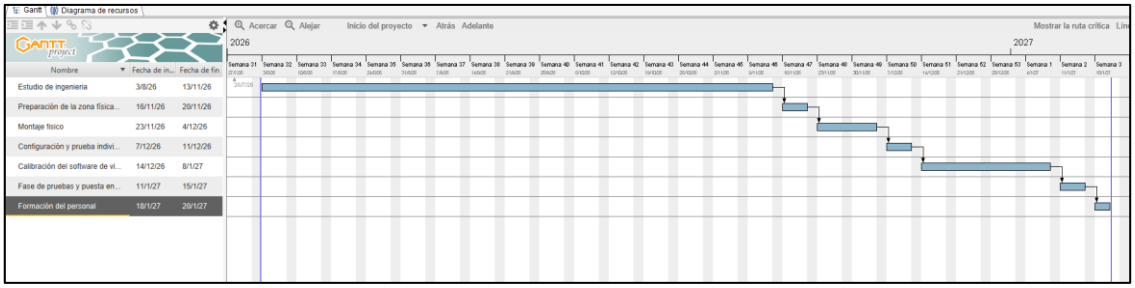


Figura 35. Diagrama de Gantt de la instalación en un entorno físico industrial

Como se puede observar en el diagrama de Gantt previo, la duración de la instalación física en total, contando con todas estas fases, estaría en torno a las 24 semanas de trabajo, ya que para la realización de cada una de las fases siguientes, es necesario que la predecesora de cada una de estas haya sido completada previamente.

4. Estudio económico

Para realizar el estudio económico de este proyecto, se han dividido los costes de implementación en dos apartados, los costes de adquisición de materiales necesarios para la instalación de la célula y los relativos a la instalación de la célula, incluyendo todos los costes de desarrollo del proyecto y los derivados del montaje físico de este.

4.1. Costes de material

Elemento	Modelo	Cantidad	Precio Unitario	Total
Brazo Robótico	UR5e	1	27.557,00 €	27.557,00 €
Cinta Motorizada	Cinta 2m	3	5.076,00 €	15.228,00 €
Mesa Rotatoria	Lewco PP360 – Turntable	1	6.400,00 €	6.400,00 €
Cinta Mesa	Cinta a medida 440 mm	1	4.500,00 €	4.500,00 €
Sensor de barrera	Wenglor P1NS101	3	200,00 €	600,00 €
Base Cobot	RB-PED-12-CB200	1	556,16 €	556,16 €
Cámara	OnRobot Eyes	1	7.568,00 €	7.568,00 €
Vallado	Panel malla FLEX II 1400 x 1800	7	98,59 €	690,13 €
Postes Vallado	Poste Paso FLEX II 1400	12	60,28 €	723,36 €
Postes Esquinas	Poste Esquina FLEX II 1400	4	60,28 €	241,12 €
Puertas Vallado	Puertas FLEX II 1400 X 1000	3	443,02 €	1.329,06 €
Licencias y Software	Licencia RoboDK anual	1	4.000,00 €	4.000,00 €
Elementos de montaje	Cableado y tornillería	1	2.000,00 €	2.000,00 €
			Precio Total	71.392,83 €

Tabla 4. Costes de adquisición de todos los materiales necesarios para la célula robotizada

4.2. Costes de ingeniería e instalación

Para llevar a cabo el cálculo de los costes derivados de la instalación del proyecto en una planta industrial, este se ha realizado utilizando los tiempos estimados en el apartado 3.10.2, calculando así el precio asociado a cada una de las fases y obteniendo el precio total de la instalación en planta (Tabla 5).

Descripción de la fase	Horas Estimadas	Precio €/h	Precio Total
Estudio de ingeniería	500	60,00 €	30.000,00 €
Preparación de la zona física para la instalación de la estación	40	45,00 €	1.800,00 €
Montaje físico de la estación	80	45,00 €	3.600,00 €
Configuración y prueba de los equipos individualmente	40	55,00 €	2.200,00 €
Calibración física del software de visión y programación del robot física	140	60,00 €	8.400,00 €
Fase de pruebas y puesta en marcha	40	55,00 €	2.200,00 €
Formación del personal	24	50,00 €	1.200,00 €
		Total	49.400,00 €

Tabla 5. Costes de montaje en físico de la célula robotizada

Costes	Precio
Compra de materiales	71.392,83 €
Ingeniería e instalación	49.400,00 €
Coste total	120.792,83 €

Tabla 6. Costes totales de la instalación de la célula

4.3. Análisis coste-beneficio

Como se puede observar en las tablas anteriores, el coste total de la instalación de este proyecto es de 120.792,83€, pero para llevar a cabo una evaluación de la viabilidad financiera del mismo es necesario realizar una comparación del rendimiento obtenido al realizar el proceso de forma manual, y al llevarla a cabo mediante el uso de la célula robotizada.

Para calcular las inspecciones por turno para cada tipo de objeto, primero se divide el total del tiempo del turno en segundos entre una suma del tiempo de inspección del objeto y 15 segundos, el tiempo estimado que tarda un objeto en avanzar hasta su posición de escaneo, obteniendo así la cantidad máxima de inspecciones que se podrían realizar en un turno asumiendo que únicamente llegaran por la cinta de entrada objetos de ese tipo. Asumiendo que los tres tipos de objetos llegan en igual cantidad, este número se divide entre 3, consiguiendo así el número de inspecciones por turno real.

Usando este número se puede obtener la mejoría porcentual en el rendimiento de la estación conseguido gracias a la implementación de esta célula automatizada (Tabla 7).

Pieza	Tiempo Operario (s)	Inspecciones por turno (operario)	Tiempo Automatizado (s)	Inspecciones por turno (auto)	Aumento inspecciones
Juguete	13	342	10	384	12,28%
Plantilla	4	505	2	564	11,68%
Taza	6	457	4	505	10,50%
				Media	11,49%

Tabla 7. Aumento de inspecciones porcentual de la célula robotizada

Como se puede observar, el aumento en la capacidad de inspecciones por turno es de un 11,49% de media, por lo que se podría aumentar la producción de la fábrica en esa cantidad como máximo. Sin embargo, esto no resulta realista, ya que se generarían más productos de los que sería posible vender para generar beneficio, además de aumentar en gran medida los costes de almacenamiento. Así, se ha tomado un valor aproximado más realista de un aumento de producción de un 5%, tomando el beneficio obtenido por objeto fabricado del fabricante de 2€ en los juguetes, de 1,50€ en las plantillas y de 1€ en las tazas. Estos beneficios son estimados, por lo que el ROI y el *payback* pueden variar dependiendo de los valores reales de estos. Así, se obtiene la siguiente tabla de beneficios (Tabla 8):

Pieza	Beneficio por pieza	Inspecciones por turno (operario)	Beneficio base	Inspecciones por turno automatizadas (aumento de producción del 5%)	Aumento de beneficio
Juguete	2,00 €	342	684,00 €	359,1	34,20 €
Plantilla	1,50 €	505	757,50 €	530,25	37,88 €
Taza	1,00 €	457	457,00 €	479,85	22,85 €
		Por turno	1.898,50 €		94,93 €
		Diario	3.797,00 €		189,85 €

Tabla 8. Beneficio por turno de la instalación robotizada con un aumento de producción del 5%

Como se puede observar, el beneficio obtenido diariamente es de 189,85€, valor que permite el cálculo del periodo de recuperación de la inversión o *payback*, el cual permite conocer la cantidad de días que tardará la empresa en recuperar la inversión inicial. Este es calculado dividiendo el coste de la inversión inicial entre el beneficio obtenido diariamente. Así, se obtiene el siguiente periodo de recuperación:

$$payback = \frac{120.792{,}83€}{189{,}85€/dia} = 636{,}25\ dias$$

Según este periodo, la inversión inicial sería recuperada al transcurrir algo más de 21 meses de producción, algo menos de 2 años, por lo que esta inversión resulta una inversión rentable en un plazo medio.

Otra métrica para tener en cuenta en este estudio económico es el Retorno de la Inversión o ROI. Este indicador mide la rentabilidad de una inversión relacionando el beneficio obtenido con el coste total obtenido para conseguirlo. Este es expresado en forma de porcentaje, lo que permite su fácil comparación con otros proyectos o inversiones. Un ROI positivo indica que la inversión genera un beneficio mayor a la inversión inicial requerida para el mismo, mientras que un ROI negativo indica que el proyecto no llega a recuperar el capital invertido. En el contexto de este proyecto, el ROI permite calcular si la inversión inicial de 120.792,83€ es compensada con el beneficio adicional generado por la célula robotizada a lo largo de sus dos primeros años de funcionamiento.

Teniendo en cuenta el beneficio diario de 189,85€ y considerando 30 días de producción por mes, se obtiene un beneficio mensual de:

$$Beneficio\ mensual = 189{,}85\,\frac{€}{dia} * 30\,\frac{dias}{mes} = \ 5695{,}50\frac{€}{mes}$$

Por lo que el beneficio acumulado durante los 24 primeros meses de funcionamiento de la célula sería:

$$Beneficio\ primeros\ 2\ años: 5695{,}50\frac{€}{mes} * 24\ meses = 136.692{,}00\ €$$

Dando como resultado el siguiente ROI:

$$ROI = \frac{136.692{,}00 - 120.792{,}83\ €}{120.792{,}83\ €} = \ 13\%$$

El resultado obtenido para este caso indica que la inversión inicial sería recuperada incluso antes de transcurrir 2 años desde su puesta en funcionamiento, por lo que representa una buena opción de inversión en el plazo previsto.

Teniendo en cuenta estos indicadores, y desde un punto de vista económico, el proyecto presenta una alta rentabilidad potencial, altamente dependiente de la capacidad de aumento de producción que pueda llevar a cabo la fábrica, y del cuello de botella que genere la inspección de sus productos en el proceso de fabricación, ya que, en el caso de un aumento del 5% de la producción, la inversión inicial es recuperada en aproximadamente 21 meses de funcionamiento y, al finalizar el segundo año, el beneficio acumulado supera el coste inicial de implementación, mientras que si se diera un aumento muy leve de la producción, esta inversión representaría un beneficio a mucho mayor plazo.

Además de los resultados económicos obtenidos, la implantación de la célula robotizada presenta una serie de ventajas adicionales que contribuyen a mejorar la viabilidad y el

rendimiento global del sistema. La automatización del proceso permite reducir la dependencia de la intervención humana en tareas repetitivas, logrando una disminución en la probabilidad de errores derivados de la capacidad de atención del operario, haciendo así más constante el rendimiento de la estación durante todo el turno. Ya que el sistema no genera fatiga de la misma forma que un operario, lo cual puede derivar en errores de clasificación y de identificación a lo largo del turno.

Por último, otro aspecto destacable es la flexibilidad proporcionada por la integración de la robótica y la visión artificial, ya que la utilización de un sistema basado en la identificación de objetos permite que la célula pueda trabajar con distintos tipos de piezas sin tener que modificar el proceso al completo, lo cual junto con el uso de herramientas de simulación y programación, permite disponer de una solución fácilmente trasladable a la instalación física, permitiendo la comprobación del funcionamiento y la ausencia de errores en este.

5. Conclusiones y líneas futuras

5.1. Conclusiones

El presente Trabajo de Fin de Grado ha tenido como objetivo el diseño, modelado y programación de una célula robotizada de inspección visual orientada al control de calidad industrial, integrando un robot colaborativo UR5e, una cámara de visión artificial OnRobot Eyes y un sistema de clasificación mediante cinta rotatoria, todo ello desarrollado y validado dentro del entorno de simulación RoboDK. A lo largo de la memoria se han abordado tanto los aspectos técnicos de diseño e implementación como los estudios normativos, de seguridad, ciberseguridad y económicos necesarios para justificar la viabilidad de una instalación de estas características en un entorno industrial real.

En primer lugar, el estudio del estado del arte realizado ha permitido contextualizar el proyecto dentro de las tendencias actuales de la automatización industrial, comprobando cómo la combinación de robótica colaborativa y visión artificial constituye una solución cada vez más extendida para tareas de control de calidad, frente a los sistemas de automatización tradicionales. La revisión de los distintos sistemas de visión artificial, tanto comerciales como basados en componentes independientes, así como de las técnicas clásicas de procesamiento de imagen, ha permitido justificar de forma fundamentada la elección de los elementos empleados en el proyecto, en particular la cámara OnRobot Eyes por su integración nativa con el ecosistema de Universal Robots.

En cuanto al desarrollo técnico de la célula, se ha conseguido diseñar un sistema en el que el robot UR5e actúa exclusivamente como soporte móvil de la cámara, en una configuración eye-in-hand que permite adquirir imágenes de cada pieza desde distintas posiciones y orientaciones, delegando la tarea de clasificación física en la cinta rotatoria. Esta separación de responsabilidades entre los distintos elementos de la célula

ha permitido simplificar la programación de cada subsistema y facilitar su depuración de forma independiente. La programación del conjunto, llevada a cabo mediante targets definidos en RoboDK y coordinada a través de Python empleando la API del simulador, ha permitido validar de forma completa el funcionamiento de la célula en un entorno virtual antes de plantear su implementación física, confirmando las ventajas del uso de herramientas de simulación y gemelos digitales.

Por otro lado, el estudio de seguridad desarrollado ha permitido aplicar de forma práctica un conjunto de normas sobre un caso concreto, comprobando que el análisis normativo no consiste en la aplicación aislada de cada norma, sino en su combinación coherente para proteger frente a los distintos peligros identificados en la instalación: desde el cálculo de distancias de seguridad en las puertas de mantenimiento mediante la fórmula de la ISO 13855, hasta la verificación del hueco lateral de la cinta transportadora frente a la tabla de aplastamiento de la ISO 13854. Este proceso ha resaltado la importancia de distinguir entre el área de trabajo máximo y el espacio de trabajo de un robot a la hora de tomar las medidas de protección, así como el papel que juegan las funciones de seguridad configurables del propio UR5e para adaptar el comportamiento del robot a las restricciones de espacio reales de una instalación.

De forma complementaria, el estudio de ciberseguridad ha permitido extender el análisis de riesgos más allá de la seguridad física de las personas, abordando la protección de la propia célula frente a posibles ataques o accesos no autorizados. La aplicación de esta serie de normas ha permitido identificar cómo un incidente de ciberseguridad podría llegar a comprometer, indirectamente, las propias funciones de seguridad física del robot y la estación, reforzando la idea de que ambos estudios, tanto de seguridad industrial como de ciberseguridad, no deben tratarse de forma independiente, sino como partes de un mismo análisis de riesgo global de la instalación.

Finalmente, el estudio económico desarrollado ha permitido comprobar la viabilidad financiera del proyecto, comparando el rendimiento de la inspección automatizada frente al proceso manual y estimando indicadores como el beneficio por turno, el retorno de la inversión y el periodo de recuperación de la misma. Estos resultados confirman que, más allá de las ventajas técnicas y de calidad que aporta la automatización del proceso de inspección, la instalación resulta también una alternativa económicamente favorable frente al proceso manual, especialmente a medio y largo plazo.

En términos generales, puede concluirse que los objetivos planteados al inicio del proyecto se han alcanzado de manera satisfactoria: se ha diseñado y validado en simulación una célula robotizada de inspección visual completa, se ha fundamentado técnica y normativamente cada una de las decisiones de diseño adoptadas, y se ha demostrado la viabilidad tanto técnica como económica de una instalación de estas características. El trabajo desarrollado constituye, además, una base sólida sobre la que podrían plantearse futuras ampliaciones o mejoras del sistema.

5.2. Posibles líneas futuras

Aunque el presente proyecto ha permitido alcanzar los objetivos planteados, su desarrollo ha puesto de manifiesto también una serie de posibles líneas de trabajo futuro que permitirían ampliar, mejorar o profundizar en distintos aspectos de la célula robotizada diseñada.

La línea de continuación más directa del proyecto sería la implementación real de la célula sobre el sistema físico, siguiendo la planificación de instalación en planta descrita en el apartado 3.10.2. Esta transición del entorno virtual al entorno real permitiría validar experimentalmente los resultados obtenidos en la simulación, ajustar los tiempos de ciclo a condiciones reales de funcionamiento y confirmar la validez de las distancias de seguridad calculadas teóricamente en el estudio de seguridad, previsiblemente introduciendo pequeños ajustes derivados de tolerancias mecánicas o de iluminación real no contempladas en el modelo virtual.

Otra línea de continuación futura sería la mejora de los algoritmos de clasificación y reconocimiento de imágenes, ya que los sistemas de visión basados en técnicas clásicas de procesamiento presentan limitaciones frente a defectos de apariencia muy variable o piezas de geometría compleja. Una posible línea de mejora consistiría en sustituir o complementar el sistema de clasificación actual mediante modelos de aprendizaje profundo (redes neuronales convolucionales) entrenados específicamente para el tipo de piezas y defectos del proyecto, lo que podría mejorar la robustez del sistema frente a variaciones no contempladas en las imágenes de referencia originales. Además, el sistema desarrollado emplea una cámara de visión 2.5D; por lo que en esta misma línea futura también se podría explorar el empleo de sistemas de visión 3D completos (luz estructurada, estereovisión) para la inspección de defectos que impliquen variaciones de altura o geometría no detectables mediante una única perspectiva, especialmente en piezas de mayor complejidad tridimensional que las contempladas en este proyecto.

Por último, el proyecto se ha centrado en el funcionamiento aislado de la célula; una línea de continuación natural sería su integración con sistemas de gestión de planta o de recursos empresariales, de forma que los resultados de la inspección, es decir, piezas aceptadas, rechazadas o tiempos de ciclo, pudieran registrarse y usarse para mejorar la trazabilidad del producto y la toma de decisiones a nivel de producción. De igual manera, en esta misma línea, y apoyándose en el gemelo digital ya desarrollado en RoboDK, podría explorarse la incorporación de un sistema de mantenimiento predictivo que, a partir de datos reales de funcionamiento del robot y de la cinta, como vibraciones, tiempos de ciclo o consumo, permitiera anticipar posibles fallos o necesidades de mantenimiento antes de que se produzca una parada no planificada de la célula, facilitando de esta forma el mantenimiento y reduciendo al mínimo la necesidad de realizar paros en la producción para el mismo.

6. Bibliografía

[1] International Federation of Robotics. World Robotics 2025 Report. IFR, Fráncfort, 2025.

[2] A. De Santis, et al. *Advances and perspectives in collaborative robotics: a review of key technologies and emerging trends*. Discover Mechanical Engineering, 2023.

[3] Carsten Steger, Markus Ulrich y Christian Wiedemann. *Machine Vision Algorithms and Applications*. Wiley-VCH, Weinheim, 2018.

[4] O. Chukwuemeka, et al. *Accuracy evaluation of hand-eye calibration techniques for vision-guided robots*. PLOS ONE, 2022.

[5] Zivid AS. *The benefits of 3D hand-eye calibration*. Zivid, 2025.

[6] G. Sharma, et al. *Edge Detection and Pattern Matching Technique for Image Analysis*. JETIR, 2023.

[7] A. Kumar, et al. *Automated Inspection System using Image Processing*. IJCRT, 2025.

[8] *CONTANDO OBJETOS (Aplicando detección de bordes con CANNY) en Python-OpenCV*, 02/01/2020. [Online]. Disponible: https://omes-va.com/contando-objetos-aplicando-deteccion-de-bordes-con-canny-en-python-opencv/. Consultado el 18/08/2026

[9] R. Silver. *Structure-guided image inspection*. United States Patent 7076093, 2006.

[10] Y. Cheng, et al. *A Comprehensive Survey for Real-World Industrial Defect Detection: Challenges, Approaches, and Prospects*. arXiv, 2025.

[11] E. Popescu, et al. *Integration of Real Signals Acquired Through External Sensors into RoboDK Simulation of Robotic Industrial Applications*. Sensors, 2025.

[12] A. Nubiola. *Key Features to Consider in Digital Twin and OLP Software*. RoboticsTomorrow, 2026.

[13] RoboDK Inc. *Simulator for Robot Arms and Offline Programming*. RoboDK, 2026.

[14] A. Owen-Hill. *Harnessing Digital Twins in Manufacturing to Transform Production and Drive Precision*. Blog RoboDK, 26/03/2025. [Online]. Disponible: https://robodk.com/blog/digital-twins-manufacturing-transform-production/. Consultado el 18/08/2026

[15] M. Popa, et al. *Integration of Machine Vision and PLC-Based Control for Scalable Quality Inspection in Industry 4.0*. PMC, 2025.

[16] OMRON Corporation. *FH Series Vision System — Features*. https://www.ia.omron.com/products/family/3210/

[17] MVTec Software GmbH. *Machine Vision by MVTec — Industry-Leading Software and Services*. MVTec, 2026.

[18] *Best Machine Vision Software in 2026: Top Tools Compared for Automation,* blog PLC Programming.io, 17/07/2026. [Online]. Disponible: https://plcprogramming.io/blog/best-machine-vision-software. Consultado el 19/08/2026

[19] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo. Robotics: Modelling, Planning and Control. Springer, 2009

[20] Hoja de especificaciones UR5e: https://www.universal-robots.com/manuals/EN/HTML/SW5_19/Content/Landingpages/Web/LandingarmUR5e.htm

[21] Cobot UR5e: https://www.universal-robots.com/mx/products/ur5e/

[22]: Normativa de seguridad EN ISO 12100. Guía para la fabricación de máquinas seguras: https://www.pilz.com/es-ES/support/law-standards-norms/iso-standards/mechanic-construction/en-iso-12100

[23] Explicación de las normas de seguridad para robots industriales: ISO 10218, ISO/TS 15066 y marcado CE (2026) https://www.evsint.com/es/industrial-robot-safety-standards-iso-10218-ce-marking-2026/

[24] Normativa de seguridad EN ISO 13855 relativa al posicionamiento de los protectores: https://www.pilz.com/es-ES/support/law-standards-norms/iso-standards/efficiency-guards/en-iso-13855

[25] Evitar el Aplastamiento de Partes del Cuerpo según EN ISO 13854: https://www.satech.it/es/en-iso-13854-espacios-minimos-seguridad-aplastamiento/

[26] Norma IEC 62443, Mejora de la ciberseguridad para los sistemas de automatización y control industrial: https://www.fortinet.com/lat/resources/cyberglossary/iec-62443

[27] Francesco Brancati, Diamantea Mongelli, Francesco Mariotti & Paolo Lollini. *A cybersecurity risk assessment methodology for industrial automation control systems,* 2025

[28] Especificación técnica IEC 63074: https://webstore.iec.ch/en/publication/69228

[29] Reglamento (UE) 2019/881 del Parlamento Europeo y del Consejo, de 17 de abril de 2019, relativo a ENISA (Agencia de la Unión Europea para la Ciberseguridad) y a la certificación de la ciberseguridad de las tecnologías de la información y la comunicación y por el que se deroga el Reglamento (UE) nº 526/2013 («Reglamento sobre la Ciberseguridad»): https://www.boe.es/buscar/doc.php?id=DOUE-L-2019-80998

[30] Librería de elementos de RoboDK: https://robodk.com/library

[31] Producto OnRobot Eyes en la web de Kawasaki Robotics: https://kawasakirobotics.com/asia-oceania/products-kaddon/onrobot-eyes/

[32] Eascan Automation INC: https://eascan.com/expertise-service/robotic-cells/

Anexos

Anexo I: Compromiso con los Objetivos de Desarrollo Sostenible:

En cumplimiento con el compromiso social de la Universitat Politècnica de València y las directrices de la Agenda 2030 de las Naciones Unidas, el presente Trabajo de Fin de Grado ha sido concebido no únicamente como un ejercicio técnico de diseño y automatización industrial, sino también como una contribución, dentro de su alcance académico, a un modelo de industria más eficiente, segura y sostenible.

Objetivo de Desarrollo Sostenible (ODS)	Alto	Medio	Bajo	No procede
Fin de la pobreza	-	-	-	X
Hambre cero	-	-	-	X
Salud y bienestar	-	-	-	X
Educación de calidad	-	-	-	X
Igualdad de género	-	-	-	X
Agua limpia y saneamiento	-	-	-	X
Energía asequible y no contaminante	-	-	-	X
Trabajo decente y crecimiento económico	X	-	-	-
Industria, innovación e infraestructuras	X	-	-	-
Reducción de las desigualdades	-	-	-	X
Ciudades y comunidades sostenibles	-	-	-	X
Producción y consumo responsables	-	X	-	-
Acción por el clima	-	-	-	X
Vida submarina	-	-	-	X
Vida de ecosistemas terrestres	-	-	-	X
Paz, justicia e instituciones sólidas	-	-	-	X
Alianzas para lograr objetivos	-	-	-	X

Tabla 9. Compromiso del proyecto con cada uno de los Objetivos de Desarrollo Sostenible (ODS)

- **ODS 8 (Trabajo decente y crecimiento económico):** El diseño de la célula, en el que el robot colaborativo asume la tarea repetitiva de posicionamiento y captura de imágenes para la inspección, contribuye a la meta de proteger los derechos laborales y promover un entorno de trabajo seguro, al liberar al personal humano de una tarea monótona y de baja aportación de valor, permitiendo reorientar su labor hacia funciones de supervisión, mantenimiento o control de calidad de mayor cualificación.
- **ODS 9 (Industria, innovación e infraestructuras):** Este proyecto contribuye directamente a la meta de modernizar la infraestructura y reconvertir las industrias para que sean sostenibles, mediante la incorporación de tecnologías propias de la Industria 4.0, como la robótica colaborativa, visión artificial, simulación mediante gemelo digital a un proceso de control de calidad tradicionalmente asociado a la inspección manual.
- **ODS 12 (Producción y consumo responsables):** La función principal de la célula desarrollada incide directamente en la meta de gestionar de manera sostenible los recursos naturales y reducir la generación de residuos. Un sistema de inspección más preciso y consistente que el realizado manualmente permite detectar defectos de forma temprana y sistemática, evitando que piezas defectuosas avancen innecesariamente por el resto del proceso productivo o lleguen al cliente final, lo que se traduce en una reducción del desperdicio de materiales, energía y trabajo invertido en productos no conformes, favoreciendo un uso más eficiente de los recursos empleados en el proceso de fabricación.